

0. Introduction to Prolog. Unification. Sicstus Prolog

In this introductory session you will get acquainted with the Prolog data types and unification rules, and

0.1 Prolog Data Types

Prolog's single data type is the *term*. Terms are either: *atoms*, *numbers*, *variables* or *compound terms (structures)*. The figure below presents a classification of the data types in Prolog:

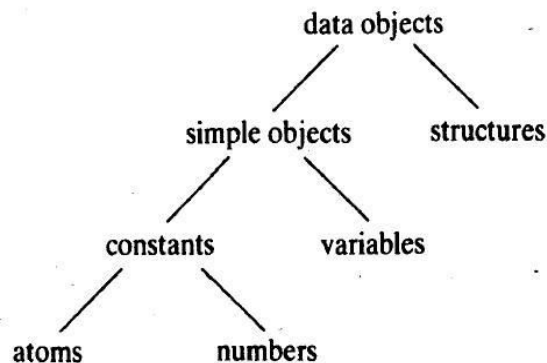


Figure 1 – Prolog data types¹

0.1.1 Atoms and numbers

An atom is a general-purpose name with no inherent meaning. It is composed of a sequence of characters that is parsed by the Prolog reader as a single unit. Atoms are usually bare words in Prolog code, written with no special syntax. However, atoms containing spaces or certain other special characters must be surrounded by single quotes. Atoms beginning with a capital letter must also be quoted, to distinguish them from variables. The empty list, written [], is also an atom. Other examples of atoms include:

x, blue, 'Taco', and 'some atom'.

Numbers can be integers or real numbers (not used very much in typical Prolog programming).

0.1.2 Variables

Variables are denoted by a string consisting of letters, numbers and underscore characters, and beginning with an upper-case letter or underscore. Variables closely

¹Ivan Bratko – *Prolog Programming for Artificial Intelligence*

resemble variables in logic in that they are placeholders for arbitrary terms. A variable can become instantiated (bound to equal a specific term) via unification. A single underscore (`_`) denotes an anonymous variable and means "any term". Unlike other variables, the underscore does not represent the same value everywhere it occurs within a predicate definition.

0.1.3 Compound Terms

A compound term is composed of an atom called a "functor" and a number of "arguments", which are again terms. Compound terms are ordinarily written as a functor followed by a comma-separated list of argument terms, which is contained in parentheses. The number of arguments is called the term's arity. An atom can be regarded as a compound term with arity zero.

Examples of compound terms are: `truck_year('Mazda', 1986)` and `'Person_Friends'(zelda,[tom,jim])`. Users can declare arbitrary functors as operators with different precedence to allow for domain-specific notations. The notation `f/n` is commonly used to denote a term with functor `f` and arity `n`.

Special cases of compound terms:

- **Lists** are defined inductively. The list `[1, 2, 3]` would be represented internally as `'(1, '(2, '(3, [])))`. A syntactic shortcut is `[H | T]`, which is mostly used to construct rules. A list can be processed by processing the first element, and then the rest of the list, in a recursive manner.

Lists can be constructed and deconstructed in a variety of ways:

- o Element enumeration: `[abc, 1, f(x), Y, g(A,rst)]`
 - o Prepending single element: `[abc | L1]`
 - o Prepending multiple elements: `[abc, 1, f(x) | L2]`
 - o Term expansion: `'(abc, '(1, '(f(x), '(Y, '(g(A,rst), []))))`
- **Strings**

0.2 Prolog Unification Rules

- **Constants** unify with themselves only:
 1. **Atoms** unify if and only if they are the same atom.
 2. **Numbers** unify if and only if they are the same number.
 3. **Strings** unify if and only if they are the same string.
 4. **Lists** unify if and only if
 - their heads unify, and
 - their tails unify.
 5. **Structures** unify if and only if

- their names unify,
- they have the same number of arguments, and
- their arguments unify.
- **Variable** V unifies with **Term** T just in case one of the following conditions is satisfied:
 - o V is an instantiated variable.
 - If T is not a variable, then V and T unify if and only if the term instantiated on V unifies with T .
 - If T is an instantiated variable, then V and T unify if and only if the term instantiated on V unifies with the term instantiated on T .
 - If T is an uninstantiated variable, then V and T are unified by instantiating on T the term that is instantiated on V .
 - o V is an uninstantiated variable.
 - If T is not a variable, then V and T are unified by instantiating T on V .
 - If T is an instantiated variable, then V and T are unified by instantiating on V the term that is instantiated on T .
 - If T is an uninstantiated variable, then V and T unify and become synonyms for the same variable.

Exercise 1.1: Check Sicstus Prolog manual for:

- a. Terms (4.1.2)
- b. Compound Terms (4.1.3)
- c. Unification (4.8.1.2)

1.2 Quiz exercises

1.2.1 Which is the nature of the following Prolog terms:

- | | | |
|---------|------------|-----------------------|
| a. X | d. hello | g. [a, b, c] |
| b. 'X' | e. Hello | h. [A, B, C] |
| c. _138 | f. 'Hello' | i. [Ana, are, 'mere'] |

1.2.2 Look up the following built-in predicates in the Prolog engine manual: `var(Term)`, `nonvar(Term)`, `number(Term)`, `atom(Term)`, `atomic(Term)` – section 4.8.1.1 *Type Checking* in Sicstus, or section 4.6 *Verify the Type of Terms* in SWI (online documentation). Test the validity of your answers from exercise 1.3.2, by using them.

1.2.3 Execute the following unification queries. Explain the results in a text file:

- a. ?- a = a.
- b. ?- a = b.
- c. ?- 1 = 2.
- d. ?- 'ana' = 'Ana'.
- e. ?- X = 1, Y = X.
- f. ?- X = 3, Y = 2, X = Y.
- g. ?- X = 3, X = Y, Y = 2.
- h. ?- X = ana.
- i. ?- X = ana, Y = 'ana', X = Y.
- j. ?- a(b,c) = a(X,Y).
- k. ?- a(X,c(d,X)) = a(2,c(d,Y)).
- l. ?- a(X,Y) = a(b(c,Y),Z).
- m. ?- tree(left, root, Right) = tree(left, root, tree(a, b, tree(c, d, e))).
- n. ?- k(s(g),t(k)) = k(X,t(Y)).
- o. ?- father(X) = X.
- p. ?- loves(X,X) = loves(marsellus,mia).
- q. ?- [1, 2, 3] = [a, b, c].
- r. ?- [1, 2, 3] = [A, B, C].
- s. ?- [abc, 1, f(x) | L2] = [abc|T].
- t. ?- [abc, 1, f(x) | L2] = [abc, 1, f(x)].

1. Facts and rules. Family tree

In this session you will learn how to write a Prolog program, and how to call Prolog predicates. Prolog employs a logic paradigm, which is inherently *declarative*. This means that the program has to specify WHAT the computer must do to solve a task, and not how (as opposed to imperative programming, in which a program contains many control statements - the HOW). As discussed at the course, a Prolog program consists of a series of predicates. Each predicate consists of (one or more) facts and/or rules.

We will start by a simple example – the genealogy tree. We will define a series of relationships – *father, mother, sibling, brother, sister, aunt, uncle, grandmother, grandfather* and *ancestor*. Let's begin with a (partial) depiction of such a tree:

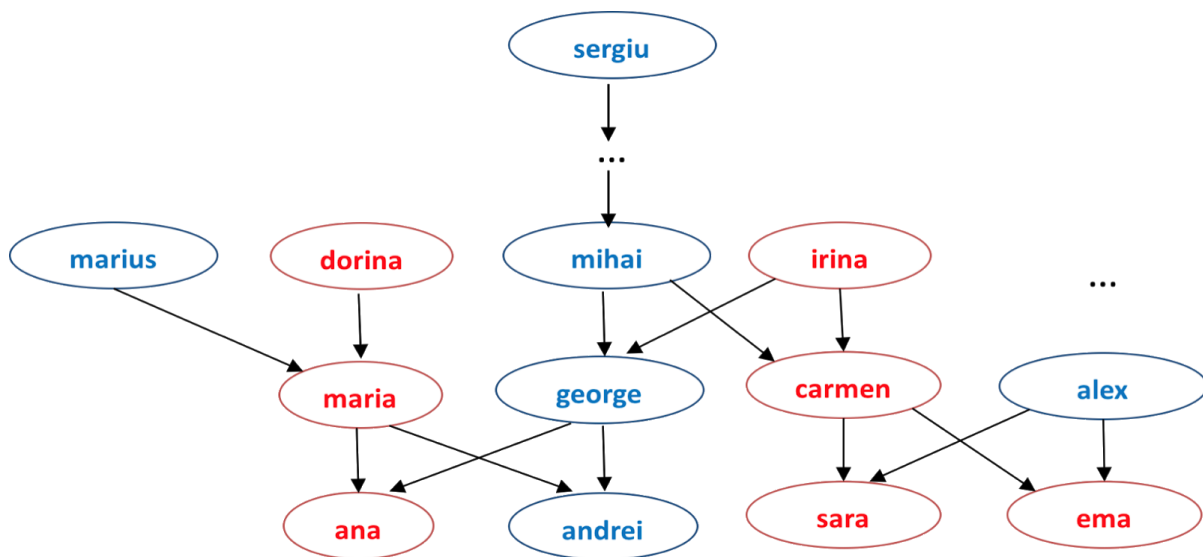


Figure 2.1 – Genealogy tree example

We will build our predicate base incrementally. So, let's start by declaring a number of facts – i.e. ground truths.

Hint: Predicate specifications are written in a source file (a simple text file). You may save it with the extension `.pl` (recognized by most Prolog engines, such as Sicstus, or SWI Prolog – a free version of Prolog). A line comment starts with `%`. Block comments are enclosed between `/* ... */`.

woman(ana). % Remember, predicate names are constant (start with lowercase letter)
woman(sara).
woman(ema).
woman(maria).
% etc...
man(andrei).
man(george).
man(alex).

```
%etc...
parent(maria, ana). % maria is ana's parent
parent(george, ana). % george also is ana's parent
parent(maria, andrei).
parent(george, andrei).
```

Therefore, we have defined three predicates: man/1, woman/1 and parent/2, each consisting of a series of facts. Predicate parent(X,Y) is to be interpreted as *X is the parent of Y*.

Now let's define a predicate for the *mother* relationship. Of course, we need two arguments – i.e. mother/2 – and we will employ the parent/2 and woman/1 predicates:

```
mother(X,Y):-woman(X), parent(X,Y).
% X is Y's mother, if X is a woman and X is the parent of Y
```

Exercise 2.1: Define predicate father/2.

Let us now call the predicates. To let Sicstus know about your defined predicates, you have to use the consult/1 built-in predicate – which you can also access directly from the *File->Consult* menu. In the file dialog, select your source file. If all goes well (i.e. not syntax errors in your source file), you should see something like this:

```
| ?- consult('E:/Scohol/Catedra/PL/Problems/genealogy.pl').
% consulting e:/scohol/catedra/pl/problems/genealogy.pl...
% consulted e:/scohol/catedra/pl/problems/genealogy.pl in module user, 0 msec 1248
bytes
| ?-
```

Calling a predicate in Prolog is known as “*asking a question*”. Below you may find a listing of Prolog queries, with the answers provided by the engine, on the predicates defined so far (do not copy-paste this in the console, it will not work):

```
| ?- man(george). % is george a man?
yes
| ?- man(X). % who is a man?
X = andrei ? ; % use ; or n to repeat the question and ask for another answer
X = george ? ;
X = alex ? ;
no
| ?- parent(X, andrei). % who are andrei's parents?
X = maria ? ;
X = george ? ;
no
| ?- parent(maria, X). % who are maria's children?
X = ana ? ;
X = andrei ? ;
no
| ?- mother(ana, X). % who are ana's children?
no
```

| ?- mother(X, ana). *% who is ana's mother?*

X = maria ? ; *% repeat the question, i.e. does ana have another mother besides maria?*

no

Ok, now try the above queries on your own in the interpreter.

Exercise 2.2: Complete the predicates man/1, woman/1 and parent/2, to have the entire genealogy tree in *Fig. 2.1* covered.

Exercise 2.3: Re-consult your source file, and execute the following queries:

- a. ?- father(alex, X).
- b. ?- father(X,Y).
- c. ?- mother(dorina, maria).

Let us now extend the predicate base with several other predicates:

% sibling/2: X and Y are siblings if they have a common parent, and they are different

sibling(X,Y):-parent(Z,X), parent(Z,Y), X\=Y.

% sister/2: X is Y's sister if X is a woman and X and Y are siblings

sister(X,Y):-sibling(X,Y), woman(X).

% aunt/2: X is Y's aunt if she is the sister of Z, who is a parent for Y.

aunt(X,Y):-sister(X,Z),parent(Z,Y).

Exercise 2.4: Extend the predicate base with predicates brother/2, uncle/2, grandmother/2 and grandfather/2.

Exercise 2.5: Re-consult your source file, and trace the execution of the following queries (by repeating the question):

- a. ?- aunt(carmen, X).
- b. ?- grandmother(dorina,X).
- c. ?- grandfather(X,ana).

Hint: to activate the trace option, simply type ?-trace. in the query prompt. You will be able to follow the execution of your queries call by call. To deactivate it, use ?-notrace.

Last, let us focus on writing a predicate ancestor/2: X is the ancestor of Y if it is linked to Y via a certain number of parent relations. In *Fig. 2.1*, *sergiu* is an ancestor of *mihai*, *sergiu*, *andrei*, *carmen* and *sara*.

Exercise 2.6: Write the ancestor/2 predicate, and execute several queries on it to test its correctness.

Introducere în Prolog

1 Obiective

Lucrarea de față vă familiarizează cu principalele concepte din limbajul Prolog: fapte, reguli și întrebări. Pe lângă acestea se vor prezenta: tipurile de date utilizate în Prolog și regulile de unificare.

2 Considerații teoretice

2.1 Paradigma logică

Paradigma de programare este un stil fundamental de programare care dictează:

- Modul de **reprezentare** a datelor (ex: fapte, variabile, clase etc.)
- Modul de **prelucrare** a datelor (ex: asignări, comparații, evaluări etc.)

Paradigma logică face parte din paradigma declarativă. Un limbaj de programare declarativ răspunde la întrebarea **CE** trebuie să facă un program. Să considerăm următorul exemplu în limbajul declarativ SQL:

```
SELECT nume, prenume FROM Studenți WHERE an = 3
```

Instrucțiunea de mai sus nu îi spune interpretorului SQL cum să realizeze căutarea, ci îi spune doar ce condiție trebuie să respecte rezultatul. Prolog este un limbaj de programare logic, deci implicit și declarativ.

În opoziție, un limbaj de programare imperativ (ex: C, Pascal, C++, C#, Java etc.) răspunde la întrebarea **CUM** trebuie să rezolve programul o anumită problemă.

2.2 Concepte Prolog

Instrucțiunile în Prolog sunt reprezentate de fapte și reguli. După ce programul sursă Prolog (*.pl) a fost consultat de interpretor (meniu *File -> Consult*) se pot pune întrebări. Răspunsul la întrebări se determină pe baza faptelor și a regulilor folosind tehnica backtracking-ului.

Tot ceea ce nu este cunoscut sau nu poate fi demonstrat este considerat fals.

Comentariile pe o linie încep cu simbolul %, pe mai multe linii se încadrează între /* și */

2.3 Fapte

Faptele sunt predicate care sunt tot timpul adevărate. Se mai numesc și axiome (din matematică). Pot avea zero, unu sau mai multe argumente. Aritatea unui predicat reprezintă numărul de argumente al acelui predicat.

Exemple:

```
suntem_in_sala_108.  
animal(elefant).  
animal('antilopa Gnu').  
inaltime(girafa, 5.5). % înălțimea îi în metrii  
arbore( t(1, t(-2, nil, nil), t(8,nil,nil)) ).
```

2.4 Tipuri de date

Simple:

- constante
 - numere (ex: 47, 6.3 etc.)
 - simboluri/atom (ex: girafa, 'Romania', 'antilopa Gnu' etc.)
 - Dacă au caractere speciale trebuie înconjurate de ghilimele simple
- variabile
 - Încep cu litera mare sau cu _ (ex: X, Aux, _12 etc.)
 - Un singur _ reprezintă o variabilă anonimă/liberă

Compuse:

- structuri (ex: t(1, t(-2, nil, nil), t(8,nil,nil)) etc.)
 - liste (ex: [], [1, 2, 3], [1, 2 | _] etc.)
 - [] = lista vidă
 - Lista [1,2,3] este reprezentată intern prin '.(1, '.(2, '.(3, []))
 - Șablonul [H|T] împarte lista în H=elementul din capul listei și T=lista ce conține restul elementelor
 - string-uri (ex: "Hello World")

2.5 Întrebări

Întrebările pot fi văzute ca și scopuri ale programului Prolog. Răspunsul la o întrebare poate fi afirmativ sau negativ. Dacă folosim variabile în întrebare putem obține și alte informații.

Exemple:

```
?- suntem_in_sala_108.  
true.
```

```
?- animal('antilopa Gnu').
true.
?- animal(X).
X = elefant ; % repetă întrebarea cu ; sau n sau spațiu
X = 'antilopa Gnu' ;
false. % nu mai există un alt animal definit în program
```

2.6 Unificare

Simbolul = realizează unificarea dintre cei 2 termeni.

Valoare este orice tip de dată care nu este/nu conține o variabilă neinițializată.

Termen 1	Termen 2	Operație
valoare	valoare	comparație
valoare	variabila neinițializată	asignare
variabila neinițializată	valoare	asignare
variabila neinițializată	variabila neinițializată	variabilele devin sinonime

2.7 Reguli

Regula reprezintă definiția unui predicat sub forma clauzei de tip Horn:

$p(\dots)$ dacă $p_{11}(\dots)$ și $p_{12}(\dots)$ și ... și $p_{1n}(\dots)$.

...

$p(\dots)$ dacă $p_{m1}(\dots)$ și $p_{m2}(\dots)$ și ... și $p_{mk}(\dots)$.

$p(\dots)$ = capul clauzei (maxim un predicat)

$p_{11}(\dots)$ și ... = corpul clauzei (zero, unu sau mai multe predicate)

fapt = clauză fără corp

întrebare = clauză fără cap

Simboluri speciale în Prolog:

`:-` = „dacă”

`,` = „și”

`;` = „sau” (poate fi realizat și prin scrierea mai multor reguli pentru același predicat)

Exemple:

```
mai_inalt(X,Y) :- inaltime(X,Hx), inaltime(Y,Hy), Hx>Hy.
% prima data luam înălțimea lui X, apoi luam înălțimea lui Y
% și în final verificăm inegalitatea
```

```
drum(X,Y) :- muchie(X,Y).
drum(X,Y) :- muchie(X,Intermediar), drum(Intermediar,Y).
% drumul între nodurile X și Y poate să fie conexiunea directă
% între cele 2 noduri SAU dacă nu există conexiune directă ne
```

% folosim de o conexiune indirectă printr-un nod intermediar

3 Exerciții

1. Ce tip de date sunt următorii termeni:

- | | | | | | |
|----|------|----|---------|----|--------------------|
| a. | X | d. | hello | g. | [a, b, c] |
| b. | 'X' | e. | Hello | h. | [A, B, C] |
| c. | _138 | f. | 'Hello' | i. | [Ana, are, 'mere'] |

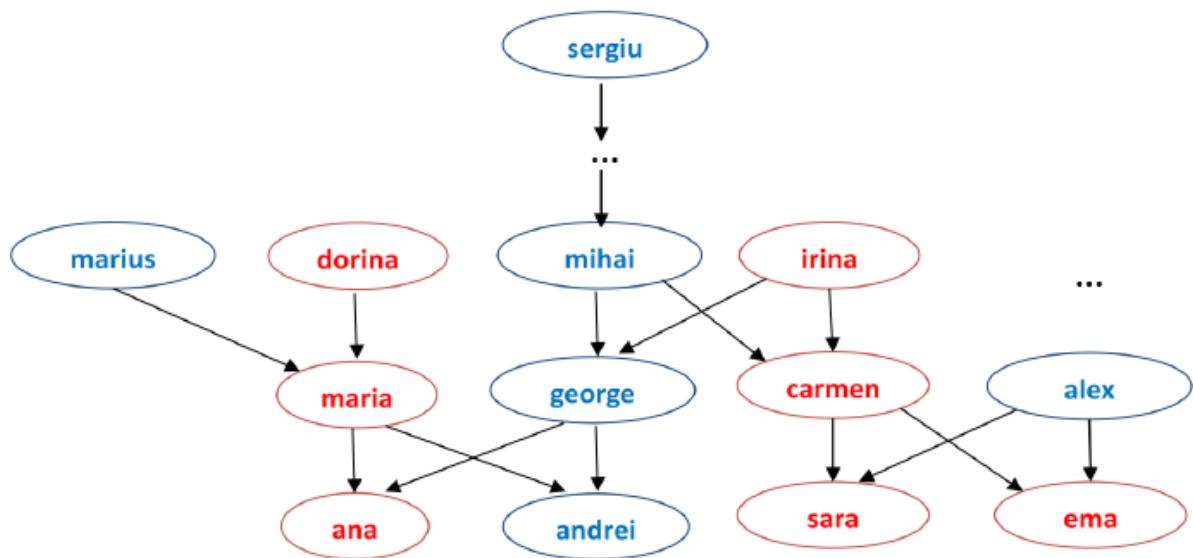
2. Ce fac următoarele predicate predefinite în Prolog:

```
var(Term),  
nonvar(Term),  
number(Term),  
atom(Term),  
atomic(Term)
```

3. Testează următoarele unificări

- a. ?- a = a.
- b. ?- a = b.
- c. ?- 1 = 2.
- d. ?- 'ana' = 'Ana'.
- e. ?- X = 1, Y = X.
- f. ?- X = 3, Y = 2, X = Y.
- g. ?- X = 3, X = Y, Y = 2.
- h. ?- X = ana.
- i. ?- X = ana, Y = 'ana', X = Y.
- j. ?- a(b,c) = a(X,Y).
- k. ?- a(X,c(d,X)) = a(2,c(d,Y)).
- l. ?- a(X,Y) = a(b(c,Y),Z).
- m. ?- tree(left, root, Right) = tree(left, root, tree(a, b, tree(c, d, e))).
- n. ?- k(s(g),t(k)) = k(X,t(Y)).
- o. ?- father(X) = X.
- p. ?- loves(X,X) = loves(marsellus,mia).
- q. ?- [1, 2, 3] = [a, b, c].
- r. ?- [1, 2, 3] = [A, B, C].
- s. ?- [abc, 1, f(x) | L2] = [abc | T].
- t. ?- [abc, 1, f(x) | L2] = [abc, 1, f(x)].

4. Scrieți noi predicate pentru relațiile de rudenie.



Figură 1. Arbore genealogic

% Predicatul woman/1

```
woman(ana).
woman(sara).
woman(ema).
woman(maria). % ...
```

% Predicatul man/1

```
man(andrei).
man(george).
man(alex). % ...
```

% Predicatul parent/2

```
parent(maria, ana). % maria este părintele anei
parent(george, ana). % george este părintele anei
parent(maria, andrei).
parent(george, andrei). % ...
```

% Predicatul mother/2

```
mother(X,Y) :- woman(X), parent(X,Y).
% X este mama lui Y, daca X este femeie și X este părintele lui Y
```

Programul sursa (ex: genealogy.pl) poate fi consultat folosind meniul din interpretor: File -> consult sau se poate scrie în interpretor următorul predicat predefinit:

```
?- consult('C:/Users/student/Desktop/genealogy.pl').
true. % Dacă nu există nici o eroare va returna true
```

4.1. Testați următoarele întrebări:

```
?- man(george). % este george bărbat?
true.
?- man(X). % cine este bărbat?
X = andrei ? ; % repetăm întrebarea cu ; sau n sau spațiu
X = george ? ;
X = alex ? ;
false.

?- parent(X, andrei). % Cine sunt părinții lui andrei?
X = maria ? ;
X = george ? ;
false.
?- parent(maria, X). % Cine sunt copiii mamei?
X = ana ? ;
X = andrei ? ;
false.

?- mother(ana, X). % Cine sunt copiii mamei?
false.
?- mother(X, ana). % Care este mama mamei?
X = maria ? ; % repetăm întrebarea = mai are ana o alta mamă?
false.
```

4.2. Scrieți predicatul father/2.

4.3. Completați predicatele man/1, woman/1 și parent/2 pentru a acoperi arborele genealogic de mai sus.

4.4. Testați următoarele întrebări:

```
?- father(alex, X).
?- father(X, Y).
?- mother(dorina, maria).
```

4.5. Testați următoarele predicate:

% Predicatul sibling/2

% X și Y sunt frați/surori dacă au același părinte și X diferit de Y
sibling(X,Y) :- parent(Z,X), parent(Z,Y), X\=Y.

% Predicatul sister/2

% X este sora lui Y dacă X este femeie și X și Y sunt frați/surori
sister(X,Y) :- sibling(X,Y), woman(X).

% Predicatul aunt/2

% X este mătușa lui Y dacă este sora lui Z și Z este părintele lui Y
aunt(X,Y) :- sister(X,Z), parent(Z,Y).

4.6. Scrieți predicatele `brother/2`, `uncle/2`, `grandmother/2` și `grandfather/2`.

4.7. Urmăriți pașii pentru găsirea răspunsului la următoarele întrebări:

?- `aunt(carmen, X)`.

?- `grandmother(dorina, Y)`.

?- `grandfather(X, ana)`.

Pentru a activa urmărirea scrieți:

?- `trace`.

Pentru a dezactiva urmărirea scrieți:

?- `notrace`.

4.8. Scrieți predicatul `ancestor/2`. X este strămoșul lui Y dacă X este legat de Y printr-o serie de relații de tip părinte.

Operații aritmetice

1 Obiective

Lucrarea de față îl familiarizează pe student cu principalele recurențe matematice și cu tipurile de recursivitate.

2 Considerații teoretice

2.1 Operatorul „is”

În Prolog expresiile matematice sunt evaluate folosind operatorul „is”. Expresia matematică se scrie în dreapta la *is* și nu poate să conțină variabile care sunt neinițializate.

Exemple:

```
?- X is sqrt(4).  
X = 2.0.
```

```
?- Y = 10, X is Y mod 2.  
X = 0.
```

```
?- X is (1+2*3)/7-1.  
X = 0.
```

2.2 Cel mai mare divizor comun (CMMDC)

Pentru a calcula cel mai mare divizor comun vom folosi algoritmul lui Euclid.
Pseudocod varianta 1 (pentru numere pozitive):

```
cmmdc(a,a) = a  
cmmdc(a,b) = cmmdc(a-b, b), dacă a>b  
cmmdc(a,b) = cmmdc(a, b-a), dacă b>a
```

Pseudocod varianta 2 (mai eficientă pentru numere mari, numere nenule):

```
cmmdc(a,0) = a  
cmmdc(a,b) = cmmdc(b, a mod b)
```

Deoarece în Prolog predicatele returnează doar adevărat/fals (yes/no), atunci va trebui să adăugăm în lista de parametrii ai predicatului și rezultatul.

% Varianta 1

```
cmmdc1(X,X,X). % parametrul 3 îi rezultatul la cmmdc  
cmmdc1(X,Y,Z) :- X>Y, Diff is X-Y, cmmdc1(Diff,Y,Z).  
cmmdc1(X,Y,Z) :- X<Y, Diff is Y-X, cmmdc1(X,Diff,Z).
```

% Varianta 2

```
cmmdc2(X,0,X). % parametrul 3 îi rezultatul la cmmdc
cmmdc2(X,Y,Z) :- Rest is X mod Y, cmmdc2(Y,Rest,Z).
```

Exemplu de urmărire a execuției (cu trace):

```
[trace] 3 ?- cmmdc1(3,3,R).
  Call: (7) cmmdc1(3, 3, _G1614) ? % se unifică cu prima clauză
  Exit: (7) cmmdc1(3, 3, 3) ? % iese cu succes
R = 3 ; % repetăm întrebarea
  Redo: (7) cmmdc1(3, 3, _G1614) ? % se unifică cu clauza 2
  Call: (8) 3>3 ? % primul apel din clauza 2
  Fail: (8) 3>3 ? % eșuează și trece la
următoarea clauză
  Redo: (7) cmmdc1(3, 3, _G1614) ? % se unifică cu clauza 3
  Call: (8) 3<3 ? % primul apel din clauza 3
  Fail: (8) 3<3 ? % eșuează
  Fail: (7) cmmdc1(3, 3, _G1614) ? % eșuează (nu mai există altă
clauză cu care să se unifice)
false.
```

```
[trace] 6 ?- cmmdc1(3,5,R).
  Call: (7) cmmdc1(3, 5, _G1614) ? % se unifică cu clauza 2
  Call: (8) 3>5 ? % primul apel din clauza 2
  Fail: (8) 3>5 ? % eșuează și trece la
următoarea clauză
  Redo: (7) cmmdc1(3, 5, _G1614) ? % se unifică cu clauza 3
  Call: (8) 3<5 ? % primul apel din clauza 3
  Exit: (8) 3<5 ? % succes
  Call: (8) _G1693 is 5-3 ? % al doilea apel din clauza 3
  Exit: (8) 2 is 5-3 ? % succes
  Call: (8) cmmdc1(3, 2, _G1614) ? % apelul recursiv din clauza 3
  Call: (9) 3>2 ? % primul apel din clauza 2
  Exit: (9) 3>2 ? % succes
  Call: (9) _G1696 is 3-2 ? % al doilea apel din clauza 2
  Exit: (9) 1 is 3-2 ? % succes
  Call: (9) cmmdc1(1, 2, _G1614) ? % apelul recursiv din clauza 3
  Call: (10) 1>2 ? % etc.
  Fail: (10) 1>2 ?
  Redo: (9) cmmdc1(1, 2, _G1614) ?
  Call: (10) 1<2 ?
  Exit: (10) 1<2 ?
  Call: (10) _G1699 is 2-1 ?
  Exit: (10) 1 is 2-1 ?
  Call: (10) cmmdc1(1, 1, _G1614) ?
  Exit: (10) cmmdc1(1, 1, 1) ?
  Exit: (9) cmmdc1(1, 2, 1) ?
  Exit: (8) cmmdc1(3, 2, 1) ?
  Exit: (7) cmmdc1(3, 5, 1) ?
R = 1
```


Urmărește execuția la:

?- cmmdc1(30,24,X).

?- cmmdc1(15,2,X).

?- cmmdc1(4,1,X).

2.3 Factorial

Recurența matematică pentru factorial este:

$\text{factorial}(0) = 1$

$\text{factorial}(n) = n * \text{factorial}(n-1)$, pentru $n > 0$

2.3.1 Recursivitatea înapoi (backwards)

În cadrul recursivității înapoi, rezultatul este construit la întoarcerea din recursivitate. Ceea ce înseamnă că inițializarea rezultatului se face după apelul recursiv.

$\text{fact1}(0,1).$

$\text{fact1}(N,F) :- N1 \text{ is } N-1, \text{fact1}(N1,F1), F \text{ is } N * F1.$

Urmărește execuția la:

?- $\text{fact1}(6, 720).$

?- $N=6, \text{fact1}(N, 120).$

?- $\text{fact1}(6, F).$

?- $\text{fact1}(N,720).$

?- $\text{fact1}(N,F).$

Dacă se repetă întrebarea, va trebui să ne asigurăm că N nu ajunge să fie negativ.

Clauza a doua din predicat ar trebui să fie:

$\text{fact1}(N,F) :- N > 0, N1 \text{ is } N-1, \text{fact1}(N1,F1), F \text{ is } N * F1.$

2.3.2 Recursivitate înainte (forward)

În cadrul recursivității înainte, rezultatul este construit într-un acumulator. Valoarea acumulatorului trebuie asignată rezultatului în ultimul pas de recursivitate (clauza de terminare). Valoarea noului acumulator se va calcula înainte de apelul recursiv.

$\text{fact2}(0,Acc,F) :- F = Acc.$

$\text{fact2}(N,Acc,F) :- N > 0, N1 \text{ is } N-1, Acc1 \text{ is } Acc * N, \text{fact2}(N1,Acc1,F).$

$\text{fact2}(N,F) :- \text{fact2}(N,1,F).$ % acumulatorul este inițializat cu 1

3 Exerciții

1. Scrieți un predicat care să calculeze cel mai mic multiplu comun (CMMMC). (Sugestie: cmmmc între două numere naturale este raportul dintre produsul lor și cmmdc)
2. Scrieți predicatul pentru ridicarea la putere în cele două tipuri de recursivitate.
3. Scrieți predicatul care calculează al n -lea număr din șirul lui Fibonacci. Formula de recurență este:

$$\text{fib}(0) = 0$$

$$\text{fib}(1) = 1$$

$$\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2), \text{ pentru } n > 0$$

4. (*) Scrieți predicatul anterior folosind doar un singur apel recursiv.
5. Scrieți predicatul *triangle/3* care verifică dacă cei trei parametri pot reprezenta lungimile laturilor unui triunghi. Suma oricăror două laturi trebuie să fie mai mare decât a treia latură.
6. Scrieți un predicat (*solve/4*) care să rezolve ecuația de gradul doi ($a \cdot x^2 + b \cdot x + c = 0$). Predicatul trebuie să aibă 3 parametri de intrare (A, B, C) și un parametru de ieșire (X). La repetarea întrebării trebuie să se obțină a doua soluție (distinctă) dacă există.

Operații pe liste

1 Obiective

Lucrarea de față îl familiarizează pe student cu operațiile ce se pot realiza pe liste: adăugarea, ștergerea, căutarea și înlocuirea de elemente.

2 Considerații teoretice

2.1 Reprezentarea unei liste

Lista vida este reprezentată prin simbolul `[]`

Lista care conține cel puțin un element poate fi reprezentată prin șablonul `[H|T]`, unde `H` este capul listei și poate fi orice tip, iar `T` este coada listei și trebuie să fie la rândul ei o listă.

Exemple:

- ?- `L=[1,2,3]`.
- ?- `[1,2,3]=[1|[2|[3]]]`.
- ?- `L=[a,b,c]`.
- ?- `L=[a, [b, [c]]]`.
- ?- `L=[a, [b], [[c]]]`.

2.2 Predicatul „member”

Predicatul *member*(*X*,*L*) verifică dacă un elementul *X* este inclus lista *L*. Recurența matematică poate fi formulată în felul următor:

$$x \in L \Leftrightarrow x = \text{primul}(L) \vee x \in \text{coada}(L)$$

În Prolog se va scrie:

```
member(X, [H|T]) :- H=X.  
member(X, [_|T]) :- member(X, T).
```

Prima clauza din predicatul *member* poate fi simplificată prin înlocuirea lui *H* din capul clauzei cu *X*. În a doua clauză, variabila *H* nu este folosită și atunci o putem înlocui cu `_` (simbolul pentru o variabilă anonimă). Aceeași înlocuire o putem face și pentru *T* din prima clauză. Astfel predicatul *member* devine:

```
member(X, [X|_]).  
member(X, [_|T]) :- member(X, T).
```

Pentru a urmări execuția predicatului *member* va trebui să-l redenumiți (ex: *member1*) deoarece acest predicat este predefinit în interpretorul Prolog.

Exemplu de urmărire a execuției (cu trace):

```
[trace] 3 ?- member1(3,[1,2,3,4]).
  Call: (7) member1(3, [1,2,3,4]) ? % se unifică cu clauza 2
  Call: (8) member1(3, [2,3,4]) ? % apelul recursiv din clauza 2
  Call: (9) member1(3, [3, 4]) ? % se unifică cu clauza 1
  Exit: (9) member1(3, [3, 4]) % succes și iese din apelul
recursiv
  Exit: (8) member1(3, [2, 3, 4]) ?
  Exit: (7) member1(3, [1, 2, 3, 4]) ?
true ; % repetăm întrebarea
  Redo: (9) member1(3, [3, 4]) ? % ultima unificare (cea cu
clauza 1) se va relua și se va face unificarea cu clauza 2
  Call: (10) member1(3, [4]) ? % se unifică cu clauza 2
  Call: (11) member1(3, []) ? % apelul recursiv din clauza 2
  Fail: (11) member1(3, []) ? % eșuează (nu există clauză
care să aibă în al doilea parametru lista vidă
  Fail: (10) member1(3, [4]) ?
  Fail: (9) member1(3, [3, 4]) ?
  Fail: (8) member1(3, [2, 3, 4]) ?
  Fail: (7) member1(3, [1, 2, 3, 4]) ?
false.
```

Urmărește execuția la:

```
?- member1(a,[a, b, c, a]).
?- X=a, member1(X, [a, b, c, a]).
?- member1(a, [1,2,3]).
```

Exemplu de comportament nedeterminist la predicatul *member*:

```
?- member1(X, [1,2,3]).
X = 1 ; % la repetarea întrebării se alege următoarea soluție
posibila
X = 2 ;
X = 3 ;
false. % nu mai exista alte soluții

?- member1(1, L).
L = [1|_] ; % prima soluție este o listă care începe cu 1
L = [_, 1|_] ; % soluția doi este o listă care are 1 pe poziția 2
L = [_, _, 1|_] % etc.
```

2.3 Predicatul „append”

Predicatul *append(L1, L2, R)* realizează concatenarea listelor *L1* și *L2* și pune rezultatul în parametrul *R*. Recurența matematică poate fi formulată în felul următor:

$$L_1 \oplus L_2 = R \Leftrightarrow \text{primul}(R) = \text{primul}(L_1) \wedge \text{coada}(R) = \text{coada}(L_1) \oplus L_2$$

În cazul în care L1 este lista vida atunci R va fi egal cu L2. În Prolog se va scrie:

```
append([], L2, R) :- R=L2.  
append([H|T], L2, R) :- append(T, L2, Coadar), R=[H|Coadar].
```

Putem simplifica predicatul prin înlocuirea parametrului R din capul celor 2 clauze cu ultima unificare.

```
append([], L2, L2).  
append([H|T], L2, [H|Coadar]) :- append(T, L2, Coadar).
```

Exemplu de urmărire a execuției (cu trace):

```
[trace] 6 ?- append1([a,b],[c,d],R).  
    Call: (7) append1([a, b], [c, d], _G1680) ?    % se unifică cu  
clauza 2 -> apoi apel recursiv  
    Call: (8) append1([b], [c, d], _G1762) ?      % se unifică cu  
clauza 2 -> apoi apel recursiv  
    Call: (9) append1([], [c, d], _G1765) ?      % se unifică cu  
clauza 1  
    Exit: (9) append1([], [c, d], [c, d]) ?      % succes și iese din  
apelul recursiv  
    Exit: (8) append1([b], [c, d], [b, c, d]) ?  
    Exit: (7) append1([a, b], [c, d], [a, b, c, d]) ?  
R = [a, b, c, d]. % nu mai exista o alta soluție
```

Exemplu de comportament nedeterminist la predicatul *append*:

```
?- append1(L1, L2, [1,2,3]).  
L1 = [],  
L2 = [1, 2, 3] ;      % prima soluție  
L1 = [1],  
L2 = [2, 3] ;        % a doua soluție  
L1 = [1, 2],  
L2 = [3] ;           % a treia soluție  
L1 = [1, 2, 3],  
L2 = [] ;            % nu mai există alte soluții  
false.
```

Urmărește execuția la:

```
?- append1([1, [2]], [3|[4, 5]], R).  
?- append1(T, L, [1, 2, 3, 4, 5]).  
?- append1(_, [X|_], [1, 2, 3, 4, 5]).
```

Inversați ordinea clauzelor din predicatul *append* și reluați trasarea întrebărilor de mai sus. Ce diferențe apar în comportamentul predicatului?

2.4 Predicatul „delete”

Predicatul *delete*(*X*, *L*, *R*) șterge prima apariție a elementului *X* din lista *L* și pune rezultatul în *R*. Dacă *X* nu există în *L* atunci *R* va fi egal cu *L*. Recurența matematică poate fi formulată în felul următor:

$$R = L - \{x\} = \begin{cases} \{\}, & L = \{\} \\ coada(L), & x = primul(L) \\ \{primul(L)\} \oplus (coada(L) - \{x\}), & altfel \end{cases}$$

În Prolog se va scrie:

```
delete(X, [X|T], T). % șterge prima apariție și se oprește
delete(X, [H|T], [H|R]) :- delete(X, T, R). % altfel iterează peste
elementele listei
delete(_, [], []). % dacă a ajuns la lista vidă înseamnă că
elementul nu a fost găsit și putem returna lista vidă
```

Urmărește execuția la:

```
?- delete1(3, [1, 2, 3, 4], R).
?- X=3, delete1(X, [3, 4, 3, 2, 1, 3], R).
?- delete1(3, [1, 2, 4], R).
?- delete1(X, [1, 2, 4], R).
```

2.5 Predicatul „delete_all”

Predicatul *delete_all*(*X*, *L*, *R*) va șterge toate aparițiile lui *X* din lista *L* și va pune rezultatul în *R*. Recurența matematică poate fi formulată în felul următor:

$$R = L - \{x\} = \begin{cases} \{\}, & L = \{\} \\ coada(L) - \{x\}, & x = primul(L) \\ \{primul(L)\} \oplus (coada(L) - \{x\}), & altfel \end{cases}$$

Predicatul *delete_all* diferă față de predicatul *delete* doar la prima clauză.

```
delete_all(X, [X|T], R) :- delete_all(X, T, R). % dacă s-a șters
prima apariție se va continua și pe restul elementelor
delete_all(X, [H|T], [H|R]) :- delete_all(X, T, R).
delete_all(_, [], []).
```

Urmărește execuția la:

```
?- delete_all(3, [1, 2, 3, 4], R).
?- X=3, delete_all(X, [3, 4, 3, 2, 1, 3], R).
?- delete_all(3, [1, 2, 4], R).
?- delete_all(X, [1, 2, 4], R).
```

3 Exerciții

1. Scrieți predicatul *append3(L1,L2,L3,R)* care să realizeze concatenarea a 3 liste.
2. Scrieți predicatul *add_first(X,L,R)* care adaugă X la începutul listei L și pune rezultatul în R.
3. Scrieți un predicat care realizează suma elementelor dintr-o lista dată.
4. Scrieți un predicat care separă numerele pare de cele impare.

```
?- separate_parity([1, 2, 3, 4, 5, 6], E, O).  
E = [2, 4, 6]  
O = [1, 3, 5] ;  
false
```

5. Scrieți un predicat care să șteargă toate elementele duplicate dintr-o listă.

```
?- remove_duplicates([3, 4, 5, 3, 2, 4], R).  
R = [3, 4, 5, 2] ; % păstrează prima apariție  
false
```

```
R = [5, 3, 2, 4] ; % păstrează ultima apariție  
false
```

6. Scrieți un predicat care să înlocuiască toate aparițiile lui X în lista L cu Y și să pună rezultatul în R.

```
?- replace_all(1, a, [1, 2, 3, 1, 2], R).  
R = [a, 2, 3, a, 2] ;  
false
```

7. (*) Scrieți un predicat care șterge tot al k-lea element din lista de intrare.

```
?- drop_k([1, 2, 3, 4, 5, 6, 7, 8], 3, R).  
R = [1, 2, 4, 5, 7, 8] ;  
false
```

Tăierea de backtracking

1 Obiective

În lucrarea de față vom prezenta o metodă de îmbunătățire a performanței unui predicat prin reducerea spațiului de căutare. De asemenea vom continua cu operații pe liste folosind cele 2 tipuri de recursivitate: înainte și înapoi.

2 Considerații teoretice

2.1 Operatorul „!”

Operatorul ! șterge ramurile alternative de căutare a soluției. O dată ce s-a trecut peste operator, apelurile din fața operatorului și predicatul inițial nu mai pot căuta o altă clauză cu care să se unifice, deci nu mai pot căuta o altă soluție.

În caz general putem scrie o clauză care folosește operatorul de tăiere în felul următor:

$p :- a_1, a_2, \dots, a_k, !, a_{k+1}, \dots, a_n.$

Dacă a_{k+1} eșuează, nu se va mai căuta o altă clauză care să se unifice cu $a_k, a_{k-1}, \dots, a_1$ și p . Un caz simplu de utilizare ar fi atunci când avem 2 clauze cu apeluri inițiale mutual exclusive.

Exemplu:

% Varianta 1 fără !

$p(X) :- X > 0, a, \dots.$

$p(X) :- X < 0, b, \dots.$

% Varianta 2 cu !

$p(X) :- X > 0, !, a, \dots.$

$p(X) :- b, \dots.$

2.1.1 Predicatul „member” cu tăiere de backtracking

$\text{member}(X, [X|_]) :- !.$

$\text{member}(X, [_|T]) :- \text{member}(X, T).$

Exemplu de urmărire a execuției (cu trace):

[trace] 12 ?- member1(X,[1,2,3]).

Call: (7) member1(_G1657, [1, 2, 3]) ?

Exit: (7) member1(1, [1, 2, 3]) ?

X = 1. % nu mai putem repeta întrebarea pentru că nu mai poate să caute o altă clauză cu care să se unifice

Urmărește execuția la:

?- X=3, member1(X, [3, 2, 4, 3, 1, 3]).

?- member1(X, [3, 2, 4, 3, 1, 3]).

2.1.2 Predicatul „delete” cu tăiere de backtracking

delete(X, [X|T], T) :- !.

delete(X, [H|T], [H|R]) :- delete(X, T, R).

delete(_, [], []).

Exemplu de urmărire a execuției (cu trace):

[trace] 15 ?- delete1(3,[4,3,2,3,1],R).

Call: (7) delete1(3, [4, 3, 2, 3, 1], _G1701) ? % se unifică cu
clauza 2 și apoi apel recursiv

Call: (8) delete1(3, [3, 2, 3, 1], _G1786) ? % se unifică cu
clauza 1

Exit: (8) delete1(3, [3, 2, 3, 1], [2, 3, 1]) ? % succes și iese
din apelul recursiv

Exit: (7) delete1(3, [4, 3, 2, 3, 1], [4, 2, 3, 1]) ?

R = [4, 2, 3, 1] ; % repetăm întrebare

Redo: (7) delete1(3, [4, 3, 2, 3, 1], _G1701) ? % nu se poate
**anula unificarea cu clauza 1 și atunci se urca în stiva, la apelul
precedent, se anulează unificarea cu clauza 2 și se încearcă
unificarea cu clauza 3**

Fail: (7) delete1(3, [4, 3, 2, 3, 1], _G1701) ? % eșec
false.

Urmărește execuția la:

?- delete1(X, [3, 2, 4, 3, 1, 3], R).

2.2 Predicatul „length”

Predicatul *length(L,R)* calculează numărul de elemente din lista *L* și pune rezultatul în *R*. Recurența matematică poate fi formulată astfel:

$$lungime(L) = \begin{cases} 0, & L = \{\} \\ 1 + lungime(coada(L)), & altfel \end{cases}$$

În Prolog se va scrie:

% Varianta 1 (recursivitate înapoi)

length1([], 0).

length1([H|T], Len) :- length1(T, Lcoada), Len is 1+Lcoada.

% Varianta 2 (recursivitate înainte)

length2([], Acc, Len) :- Len=Acc.

length2([H|T], Acc, Len) :- Acc1 is Acc + 1, length2(T, Acc1, Len).

length2_pretty(L, R) :- length2(L, 0, R).

Urmărește execuția la:

```
?- length1([a, b, c, d], Len).  
?- length1([1, [2], [3|[4]]], Len).  
?- length2([a, b, c, d], 0, Res).  
?- length2_pretty([a, b, c, d], Len).  
?- length2_pretty([1, [2], [3|[4]]], Len).  
?- length2([a, b, c, d], 3, Len).
```

2.3 Predicatul „reverse”

Predicatul *reverse(L,R)* inversează ordinea elementelor din lista *L*. Recurența matematică poate fi formulată astfel:

$$invers(L) = \begin{cases} \{\}, & L = \{\} \\ invers(coada(L)) \oplus \{primul(L)\}, & altfel \end{cases}$$

În Prolog se va scrie:

% Varianta 1 (recursivitate înapoi)

```
reverse1([], []).  
reverse1([H|T], R) :- reverse1(T, Rcoada), append1(Rcoada, [H], R).
```

% Varianta 2 (recursivitate înainte)

```
reverse2([], Acc, R) :- Acc=R.  
reverse2([H|T], Acc, R) :- Acc1=[H|Acc], reverse2(T, Acc1, R).  
reverse2_pretty(L, R) :- reverse2(L, [], R).
```

Urmărește execuția la:

```
?- reverse1([a, b, c, d], R).  
?- reverse1([1, [2], [3|[4]]], R).  
?- reverse2_pretty([a, b, c, d], R).  
?- reverse2_pretty([1, [2], [3|[4]]], R).  
?- reverse2([a, b, c, d], [1, 2], R).
```

2.4 Predicatul minim

Predicatul *minim(L,M)* determina elementul minim din lista *L*. Dacă lista este vidă va returna false. Recurența matematică poate fi formulată astfel:

$$min(L) = \begin{cases} min(coada(L)), & min(coada(L)) < primul(L) \\ primul(L), & altfel \end{cases}$$

În Prolog se va scrie:

% Varianta 1 (recursivitate înapoi)

```
min1([H|T], M) :- min1(T, M), M<H, !.  
min1([H|_], H).
```

% Varianta 2 (recursivitate înainte)

```
min2([], Mp, M) :- M=Mp.  
min2([H|T], Mp, M) :- H<Mp, !, min2(T, H, M).  
min2([H|T], Mp, M) :- min2(T, Mp, M).  
min2_pretty([H|T], M) :- min2(T, H, M). % la început inițializăm  
minimul cu primul element
```

Urmărește execuția la:

```
?- min1([], M).  
?- min1([3, 2, 6, 1, 4, 1, 5], M).  
?- min2_pretty([], M).  
?- min2_pretty([3, 2, 6, 1, 4, 1, 5], M).
```

2.5 Operații pe mulțimi

Vom reprezenta o mulțime folosind o listă fără elemente duplicate. Reuniunea între 2 mulțimi poate fi realizată prin concatenarea listei a doua cu elementele din prima listă care nu apar în a doua listă. Recurența matematică poate fi formulată astfel:

$$L_1 \cup L_2 = \begin{cases} \{\text{primul}(L_1)\} \oplus (\text{coada}(L_1) \cup L_2), & \text{primul}(L_1) \notin L_2 \\ \text{coada}(L_1) \cup L_2, & \text{altfel} \end{cases}$$

În Prolog se va scrie:

```
union([], L, L).  
union([H|T], L2, R) :- member(H, L2), !, union(T, L2, R).  
union([H|T], L2, [H|R]) :- union(T, L2, R).
```

Urmărește execuția la:

```
?-union([1,2,3], [4,5,6], R).  
?-union([1,2,5], [2,3], R).  
?-union(L1,[2,3,4],[1,2,3,4,5]).
```

3 Exerciții

1. Scrieți predicatul *inters(L1,L2,R)* care realizează intersecția între două mulțimi.
2. Scrieți predicatul *diff(L1,L2,R)* care realizează diferența între două mulțimi (elementele care apar în prima mulțime și nu apar în a doua mulțime).
3. Scrieți predicatele *del_min(L,R)* și *del_max(L,R)* care șterg toate aparițiile minimului, respectiva ale maximumului din lista *L*.
4. Scrieți un predicat care inversează ordinea elementelor dintr-o listă începând cu al K-lea element.

```
?- reverse_k([1, 2, 3, 4, 5], 2, R).  
R = [1, 2, 5, 4, 3] ;  
false
```

5. Scrieți un predicat care codifică șirul de elemente folosind algoritmul RLE (Run-length encoding). Un șir de elemente consecutive și egale se vor înlocui cu perechi [element, număr de apariții].

```
?- rle_encode([1, 1, 1, 2, 3, 3, 1, 1], R).  
R = [[1, 3], [2, 1], [3, 2], [1, 2]] ;  
false
```

6. Scrieți un predicat care rotește o listă K poziții la dreapta.

```
?- rotate_right([1, 2, 3, 4, 5, 6], 2, R).  
R = [5, 6, 1, 2, 3, 4] ;  
false
```

7. (*) Scrieți un predicat care extrage aleatoriu K element din lista *L* și le pune în lista rezultat *R*. *Sugestie: folosiți funcția random(valoare_maxima).*

```
?- rnd_select([a, b, c, d, e, f, g, h], 3, R).  
R = [e, d, a] ;  
false
```

Sortări

1 Obiective

În lucrarea de față vom prezenta mai multe metode de sortare implementate în Prolog.

2 Considerații teoretice

2.1 Sortarea prin permutări

Această metodă generează toate permutările posibile ale listei de intrare până când se ajunge la o permutare care are toate elementele ordonate și atunci se oprește.

```
perm_sort(L,R):-perm(L, R), is_ordered(R), !.
```

Numărul de permutări ale unei liste cu n elemente este egal cu $n!$. Pentru a genera permutările, vom alege un element H aleatoriu din lista de intrare și îl vom pune pe prima poziție în lista rezultat. Alegerea lui H se va realiza folosind nedeterminismul predicatului *append*. Lista de intrare poate fi scrisă ca o concatenare de 2 sub-liste. Elementul H îl alegem să fie primul element din a doua sub-listă.

```
perm(L, [H|R]):-append(A, [H|T], L), append(A, T, L1), perm(L1, R).  
perm([], []).
```

Mai avem de scris predicatul care verifică dacă lista rezultat este ordonată. Alegem ca lista rezultat să fie ordonată crescător.

```
is_ordered([H1, H2|T]):-H1 =< H2, is_ordered([H2|T]).  
is_ordered([_]). % daca ii doar un element ii deja ordonata
```

Urmărește execuția la:

```
?- append(A, [H|T], [1, 2, 3]), append(A, T, R).  
?- perm([1, 2, 3], L).  
?- is_ordered([1, 2, 4, 4, 5]).  
?- perm_sort([1, 4, 2, 3, 5], R).
```

2.2 Sortarea prin selecție

Această sortare alege la fiecare pas cel mai mic (cel mai mare) element din lista nesortată și îl mută la capătul liste deja sortate. Cel mai mic element din lista de intrare reprezintă primul element din lista rezultat.

```
sel_sort(L, [M|R]):- min(L, M), delete(M, L, L1), sel_sort(L1, R).  
sel_sort([], []).
```

Predicatele *min* și *delete* sunt de la lucrările precedente.

Urmărește execuția la:

```
?- sel_sort([3, 2, 4, 1], R).
```

2.3 Sortarea prin inserție

Această sortare extrage la fiecare pas un element din lista nesortată (de obicei primul element) și îl inserează în poziția corectă în lista sortată (folosind căutare liniară sau binară).

```
ins_sort([H|T], R):- ins_sort(T, R1), insert_ord(H, R1, R).  
ins_sort([], []).
```

```
insert_ord(X, [H|T], [H|R]):-X>H, !, insert_ord(X, T, R).  
insert_ord(X, T, [X|T]).
```

Urmărește execuția la:

```
?- insert_ord(3, [], R).  
?- insert_ord(3, [1, 2, 4, 5], R).  
?- insert_ord(3, [1, 3, 3, 4], R).  
?- ins_sort([3, 2, 4, 1], R).
```

2.4 Sortarea bulelor

Sortarea bulelor realizează mai multe treceri peste lista de intrare. La fiecare trecere verifică două câte două elemente consecutive și le interschimbă dacă nu respectă condiția de ordine. Dacă la o trecere nu s-a făcut nici o interschimbare atunci sortarea s-a terminat. Pentru a verifica dacă s-a făcut o interschimbare ne vom folosi de un *flag* (dacă variabila este inițializată înseamnă că nu am terminat).

```
bubble_sort(L,R):-one_pass(L,R1,F), nonvar(F), !, bubble_sort(R1,R).  
bubble_sort(L,L).
```

```
one_pass([H1,H2|T], [H2|R], F):-H1>H2, !, F=1, one_pass([H1|T],R,F).  
one_pass([H1|T], [H1|R], F):-one_pass(T, R, F).  
one_pass([], [] ,_).
```

Urmărește execuția la:

```
?- one_pass([1, 2, 3, 4], R, F).  
?- one_pass([2, 3, 1, 4], R, F).  
?- bubble_sort([1, 2, 3, 4], R).  
?- bubble_sort([2, 3, 1, 4], R).
```

2.5 Sortare rapidă

Sortarea rapidă se folosește de un pivot care împarte lista de intrare în două sub-liste (tehnică *divide et impera*). O sub-listă cu elementele mai mici decât pivotul și o a doua sub-listă cu elementele mai mari decât pivotul. Repetă acest proces până când sub-listele devin vide.

```
quick_sort([H|T], R):- % alegem pivot primul element
    partition(H, T, Sm, Lg),
    quick_sort(Sm, SmS), % sortam sublista cu elementele mai mici
    decât pivotul
    quick_sort(Lg, LgS), % sortam sublista cu elementele mai mari
    decât pivotul
    append(SmS, [H|LgS], R).
quick_sort([], []).

partition(H, [X|T], [X|Sm], Lg):-X<H, !, partition(H, T, Sm, Lg).
partition(H, [X|T], Sm, [X|Lg]):-partition(H, T, Sm, Lg).
partition(_, [], [], []).
```

Urmărește execuția la:

```
?- partition(3, [4, 2, 6, 1, 3], Sm, Lg).
?- quick_sort([3, 2, 5, 1, 4, 3], R).
?- quick_sort([1, 2, 3, 4], R).
```

2.6 Sortare prin interclasare

Sortarea prin interclasare împarte lista de intrare în două sub-liste de lungimi egale (tehnică *divide et impera*). Apoi sortează cele două sub-liste și în final interclasează sub-listele deja sortate.

```
merge_sort(L, R):-
    split(L, L1, L2), % împarte L în doua subliste de lungimi egale
    merge_sort(L1, R1),
    merge_sort(L2, R2),
    merge(R1, R2, R). % interclasează sublistele ordonate
merge_sort([H], [H]). % split returnează fail dacă lista ii vidă sau
are doar un singur element
merge_sort([], []).

split(L, L1, L2):-
    length(L, Len),
    Len>1,
    K is Len/2,
    splitK(L, K, L1, L2).

splitK([H|T], K, [H|L1], L2):-K>0,!,K1 is K-1,splitK(T, K1, L1, L2).
splitK(T, _, [], T).
```

```

merge([H1|T1], [H2|T2], [H1|R]):-H1<H2, !, merge(T1, [H2|T2], R).
merge([H1|T1], [H2|T2], [H2|R]):-merge([H1|T1], T2, R).
merge([], L, L).
merge(L, [], L).

```

Urmărește execuția la:

```

?- split([2, 5, 1, 6, 8, 3], L1, L2).
?- split([2], L1, L2).
?- merge([1, 5, 7], [3, 6, 9], R).
?- merge([1, 1, 2], [1], R).
?- merge([], [3], R).
?- merge_sort([4, 2, 6, 1, 5], R).

```

3 Exerciții

1. Rescrieți predicatul *perm* fără a apela predicatul *append*. Extragerea și ștergerea unui element trebuie realizate altfel.
2. Rescrieți predicatul *sel_sort* astfel încât să sorteze descrescător.
3. Scrieți un predicat care să sorteze o listă de caractere ASCII. *Sugestie: folosiți char_code*

```

?- sort_chars([e, t, a, v, f], L).
L = [a, e, f, t, v] ;
false

```

4. Scrieți un predicat care să sorteze o lista de sub-liste în funcție de lungimea sub-listelor.

```

?- sort_len([[a, b, c], [f], [2, 3, 1, 2], [], [4, 4]], R).
R = [[], [f], [4, 4], [a, b, c], [2, 3, 1, 2]] ;
false

```


Liste adânci

1 Obiective

În lucrarea de față vom prezenta o generalizare a tipului listă. Elementele listei pot să fie la rândul lor tot liste. Astfel o listă adâncă poate să aibă elemente la diverse nivele de imbricare.

Exemple:

```
L1 = [1,2,3,[4]]
L2 = [[1],[2],[3],[4,5]]
L3 = [[],2,3,4,[5,[6]],7]]
L4 = [[[1]],1,[1]]
L5 = [1,[2],[[3]],[[[4]]],[5,[6,[7,[8,[9],10],11],12],13]]
L6 = [alpha,2,[beta],[gamma,[8]]]
```

Testați următoarele întrebări:

```
?- member(2,L5).
?- member([2], L5).
?- member(X, L5).
?- append(L1,R,L2).
?- append(L4,L5,R).
?- delete(1, L4,R).
?- delete(13,L5,R).
```

2 Considerații teoretice

2.1 Predicatul „atomic”

Vom folosi predicatul *atomic(X)* pentru a verifica dacă X este un element simplu (număr, simbol) sau este o structură (ex: o altă listă).

Testați următoarele întrebări:

```
? - atomic(apple).
? - atomic(4).
? - atomic(X).
? - atomic(apple(2)).
? - atomic([1,2,3]).
? - atomic([]).
```

2.2 Predicatul „depth”

Acest predicat calculează nivelul maxim de imbricare al unei liste adânci. Iterează peste elementele listei și verifică natura lor. Dacă elementul nu este atomic atunci se apelează recursiv pe acel element. Adâncimea listei este egală cu adâncimea maximă a unui element plus unu.

```
depth([],1).
depth([H|T],R):- atomic(H), !, depth(T,R).
depth([H|T],R):- depth(H,R1), depth(T,R2), R3 is R1+1, max(R3,R2,R).
```

Testați predicatul pentru listele L1-L6 (ex: ?-depth(L1, R).)

2.3 Predicatul „flatten”

Acest predicat aplatizează o listă adâncă. Lista rezultat va conține numai elemente atomice. Și în acest caz trebuie să se verifice natura elementului. Dacă elementul este la rândul lui o listă atunci va trebui aplatizată.

```
flatten([],[]).
flatten([H|T], [H|R]):- atomic(H), !, flatten(T,R).
flatten([H|T], R):- flatten(H,R1), flatten(T,R2), append(R1,R2,R).
```

Testați predicatul pentru listele L1-L6 (ex: ?-flatten(L1, R).)

2.4 Predicatul „heads”

Acest predicat extrage toate elementele atomice de la capul fiecărei liste. Ne vom folosi de un al treilea parametru pentru a ști dacă am extras capul listei curente sau nu.

```
heads([],[],_).
```

% dacă flag=1 atunci suntem la început de lista și putem extrage capul listei; în apelul recursiv setam flag=0

```
heads([H|T],[H|R],1):- atomic(H), !, heads(T,R,0).
```

% dacă flag=0 atunci nu suntem la primul element atomic și atunci continuăm cu restul elementelor

```
heads([H|T],R,0):- atomic(H), !, heads(T,R,0).
```

% dacă am ajuns la aceasta clauza înseamnă că primul element nu este atomic și atunci trebuie să apelăm recursiv și pe acest element

```
heads([H|T],R,_):- heads(H,R1,1), heads(T,R2,0), append(R1,R2,R).
```

```
heads_pretty(L,R):- heads(L, R, 1).
```

Testați predicatul pentru listele L1-L6 (ex: ?- heads_pretty(L1, R).)

2.5 Predicatul „member”

Dacă vrem să căutam un element la alt nivel de imbricare va trebui să modificăm predicatul *member* astfel:

% Varianta 1

```
member1(H,[H|_]).
```

```
member1(X,[H|_]):-member1(X,H).
```

```
member1(X,[_|T]):-member1(X,T).
```

% Varianta 2

```
member2(X,L):- flatten(L,L1), member(X,L1).
```

Testați următoarele întrebări:

- ? - member1(1,L1).
- ? - member1(4,L2).
- ? - member1([5,[6]], L3).
- ? - member1(X,L4).
- ? - member1(X,L6).
- ? - member1(14,L5).

3 Exerciții

1. Scrieți predicatul *count_atomic(L,R)* care calculează numărul de elemente atomice din lista *L* (toate elementele atomice).
2. Scrieți predicatul *sum_atomic(L,R)* care calculează suma elementelor atomice din lista *L* (toate elementele atomice).
3. Modifică predicatul *member* pentru liste adânci astfel încât să fie determinist.
4. Scrieți predicatul *lasts(L,R)* care extrage elementele atomice de pe ultima poziție din fiecare sublistă din *L*.

```
?- lasts([1,2,[3],[4,5],[6,[7]]], R).  
R = [3,5,7] ;  
false
```

5. Scrieți predicatul *replace(X,Y,L,R)* care înlocuiește pe *X* cu *Y* în lista adâncă *L* (la orice nivel de imbricare) și pune rezultatul în *R*.
6. (*) Scrieți un predicat care să sorteze o listă adâncă în funcție de adâncimea fiecărui element. Dacă două elemente au aceeași adâncime atunci se vor compara în funcție de elementele atomice pe care le conțin.

```
?- sort_depth([[[[1]]], 2, [5,[4],7], [[5],4], [5,[0,9]]], R).  
R = [2, [5,[0,9]], [[5],4], [5,[4],7], [[[1]]]] ;  
false
```

Arbori

1 Obiective

În lucrarea de față vom prezenta operațiile pe arbori binari de căutare. În Prolog, arborii sunt reprezentați prin structuri recursive. Arborele gol îl reprezentăm prin simbolul *nil*.

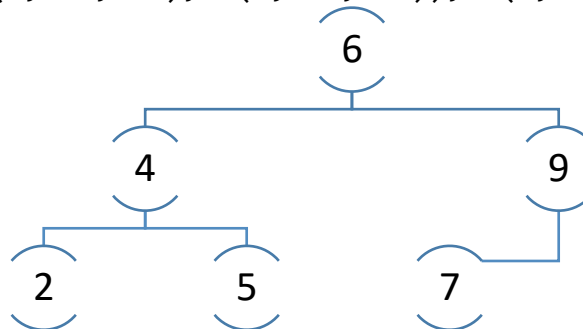
2 Considerații teoretice

2.1 Reprezentare

Arborele binar de căutare este reprezentat printr-o structură cu 3 argumente: cheia nodului curent, subarborele stâng și subarborele drept.

Exemplu:

```
t(6, t(4, t(2,nil,nil), t(5,nil,nil)), t(9, t(7,nil,nil), nil))
```



Pentru a nu scrie de fiecare dată arborele în interpretorul Prolog, puteți „salva” structura în fișier într-un fapt (axiomă).

Exemple:

```
tree1(t(6, t(4,t(2,nil,nil),t(5,nil,nil)), t(9,t(7,nil,nil),nil))).  
tree2(t(8, t(5, nil, t(7, nil, nil)), t(9, nil, t(11, nil, nil)))).
```

...

```
?- tree1(T), operatie_pe_arbore(T,...).
```

2.2 Traversarea arborelui

Există 3 posibilități de a traversa arborele în funcție de ordinea în care sunt procesate nodurile: în-ordine, pre-ordine și post-ordine.

Traversarea în-ordine colectează nodurile din subarborele stâng, apoi nodul curent și în final nodurile din subarborele drept.

```
inorder(t(K,L,R), List):-inorder(L,LL), inorder(R,LR),  
                           append(LL, [K|LR], List).  
inorder(nil, []).
```

```

% cheie, subarbore stâng și subarbore drept
preorder(t(K,L,R), List):-preorder(L,LL), preorder(R, LR),
                           append([K|LL], LR, List).
preorder(nil, []).

% subarbore stâng, subarbore drept și apoi cheia
postorder(t(K,L,R), List):-postorder(L,LL), postorder(R, LR),
                           append(LL, LR,R1), append(R1, [K], List).
postorder(nil, []).

```

Urmărește execuția la:

```

?- tree1(T), inorder(T, L).
?- tree1(T), preorder(T, L).
?- tree1(T), postorder(T, L).

```

2.3 Afișare „pretty”

Ne vom folosi de afișarea „pretty” pentru a vizualiza corectitudinea operațiilor pe arbori. Cea mai ușoară variantă de a afișa un arbore este de a-l traversa în-ordine și fiecare nod să fie afișat la un număr de spații egal cu adâncimea la care se află nodul în arbore. Rădăcina arborelui se află la adâncimea 0.

Exemplu:

```

|           2
|      4
|           5
|6
|           7
|      9

```

```

pretty_print(nil, _).
pretty_print(t(K,L,R), D):-D1 is D+1,
                           pretty_print(L, D1),
                           print_key(K, D),
                           pretty_print(R, D1).

```

```

% predicat care afișează cheia K la D tab-uri față de marginea din
% stânga și inserează o linie nouă
print_key(K, D):-D>0, !, D1 is D-1, write('\t'), print_key(K, D1).
print_key(K, _):-write(K), write('\n').
% write('\n') îi echivalent cu predicatul nl

```

Urmărește execuția la:

```

?- tree2(T), pretty_print(T, 0).

```

2.4 Căutare unei chei

Predicatul *search_key(Key, T)* verifică dacă există un nod cu cheia *Key* în arborele *T*. Recurența matematică poate fi formulată astfel:

$$\text{search}(\text{key}, T) = \begin{cases} \text{false} & T = \text{null} \\ \text{true} & T.\text{root}.\text{key} = \text{key} \\ \text{search}(\text{key}, T.\text{left}) & T.\text{root}.\text{key} < \text{key} \\ \text{search}(\text{key}, T.\text{right}) & T.\text{root}.\text{key} > \text{key} \end{cases}$$

În Prolog se va scrie:

```
search_key(Key, t(Key, _, _)):-!.
search_key(Key, t(K, L, _)):-Key<K, !, search_key(Key, L).
search_key(Key, t(_, _, R)):-search_key(Key, R).
```

Urmărește execuția la:

```
?- tree1(T), search_key(5,T).
?- tree1(T), search_key(8,T).
```

2.5 Inserarea unei chei

Primul pas constă în căutarea poziției în arbore unde trebuie inserat nodul cu cheia dată (nodul care se inserează este de tip frunză). Dacă mai există un nod cu aceeași cheie atunci nu se realizează inserarea.

```
insert_key(Key, nil, t(Key, nil, nil)). % inserează cheia în arbore
insert_key(Key, t(Key, L, R), t(Key, L, R)):-!. % cheia există deja
insert_key(Key, t(K,L,R), t(K,NL,R)):- Key<K,!,insert_key(Key,L,NL).
insert_key(Key, t(K,L,R), t(K,L,NR)):- insert_key(Key, R, NR).
```

Urmărește execuția la:

```
?- tree1(T),pretty_print(T,0),insert_key(8,T,T1),pretty_print(T1,0).
?- tree1(T),pretty_print(T,0),insert_key(5,T,T1),pretty_print(T1,0).
?- insert_key(7, nil, T1), insert_key(12, T1, T2), insert_key(6, T2,
T3), insert_key(9, T3, T4), insert_key(3, T4, T5), insert_key(8, T5,
T6), insert_key(3, T6, T7), pretty_print(T7, 0).
```

2.6 Ștergerea unei chei

În cazul ștergerii unei chei trebuie să căutăm nodul care are acea cheie și apoi trebuie să ne asigurăm că în urma ștergerii nodului respectiv, arborele va continua să fie arbore binar de căutare.

```
delete_key(Key, t(Key, L, nil), L):-!.
delete_key(Key, t(Key, nil, R), R):-!.
delete_key(Key, t(Key, L, R), t(Pred,NL,R)):-!,get_pred(L,Pred,NL).
delete_key(Key, t(K,L,R), t(K,NL,R)):-Key<K,!,delete_key(Key,L,NL).
delete_key(Key, t(K,L,R), t(K,L,NR)):-delete_key(Key,R,NR).
```

% caută nodul predecesor

```
get_pred(t(Pred, L, nil), Pred, L):-!.
```

```
get_pred(t(Key, L, R), Pred, t(Key, L, NR)):-get_pred(R, Pred, NR).
```

Urmărește execuția la:

```
?- tree1(T), pretty_print(T, 0), delete_key(5, T, T1),  
pretty_print(T1, 0).
```

```
?- tree1(T), pretty_print(T, 0), delete_key(9, T, T1),  
pretty_print(T1, 0).
```

```
?- tree1(T), pretty_print(T, 0), delete_key(6, T, T1),  
pretty_print(T1, 0).
```

```
?- tree1(T), pretty_print(T, 0), insert_key(8, T, T1),  
pretty_print(T1, 0), delete_key(6, T1, T2), pretty_print(T2, 0),  
insert_key(6, T2, T3), pretty_print(T3, 0).
```

2.7 Înălțime unui arbore

Predicatul $height(T, R)$ calculează distanța maximă între rădăcina arborelui și o frunză. Recurența matematică poate fi formulată astfel:

$$height(T) = \begin{cases} 0 & T = null \\ 1 + \max(height(T.left), height(T.right)) & altfel \end{cases}$$

În Prolog se va scrie:

```
height(nil, 0).
```

```
height(t(_, L, R), H):-height(L, H1),  
                        height(R, H2),  
                        max(H1, H2, H3),  
                        H is H3+1.
```

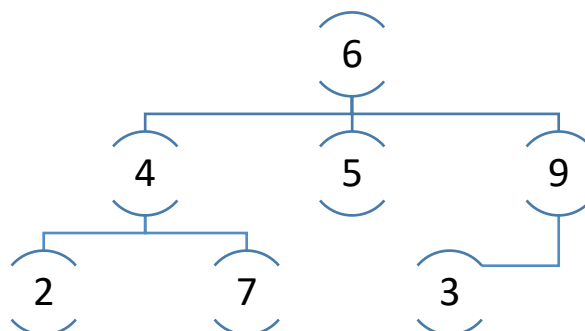
Urmărește execuția la:

```
?- tree1(T), pretty_print(T, 0), height(T, H).
```

```
?- tree1(T), height(T, H), pretty_print(T, 0), insert_key(8, T, T1),  
height(T1, H1), pretty_print(T1, 0).
```

2.8 Arbori ternari

Într-un arbore ternar un nod poate să aibă maxim 3 copii. Arborele îl putem reprezenta folosind o structură de tipul $t(Key, Left, Middle, Right)$.



3 Exerciții

1. Scrieți predicatele care iterează peste elementele unui arbore ternar
 - 1.1. $\text{inorder} = \text{Left} \rightarrow \text{Root} \rightarrow \text{Middle} \rightarrow \text{Right}$
 - 1.2. $\text{preorder} = \text{Root} \rightarrow \text{Left} \rightarrow \text{Middle} \rightarrow \text{Right}$
 - 1.3. $\text{postorder} = \text{Left} \rightarrow \text{Middle} \rightarrow \text{Right} \rightarrow \text{Root}$
2. Scrieți un predicat care calculează înălțimea unui arbore ternar.
3. Rescrieți predicatul *delete_key* folosind nodul succesor.
4. Scrieți un predicat care colectează într-o listă toate cheile din frunzele arborelui binar.
5. Scrieți un predicat care calculează diametrul unui arbore binar

$$\text{diam}(T) = \max\{\text{diam}(T.\text{left}), \text{diam}(T.\text{right}), \text{height}(T.\text{left}) + \text{height}(T.\text{right}) + 1\}$$

6. (*) Scrieți un predicat care colectează într-o listă toate nodurile de la aceeași adâncime din arborele binar.
7. (*) Verificați dacă un arbore binar este simetric. Un arbore binar este simetric dacă subarborele stâng este imaginea în oglindă a subarborelui drept. Ne interesează structura arborelui nu și valorile din noduri.

```
?- tree1(T), symmetric(T).  
false
```

```
?- tree1(T), delete_key(2, T, T1), symmetric(T1).  
T = t(6,t(4,t(2,nil,nil),t(5,nil,nil)),t(9,t(7,nil,nil),nil)),  
T1 = t(6,t(4,nil,t(5,nil,nil)),t(9,t(7,nil,nil),nil)) ? ;  
false
```


Structuri incomplete

1 Obiective

În lucrarea de față vom prezenta un caz special de structuri. Structurile incomplete nu se termină într-o constantă (ex: [] pentru liste sau *nil* pentru arbori), ci se termină într-o variabilă liberă „_”.

2 Considerații teoretice

2.1 Reprezentare

Structurile incomplete (structuri instanțiate parțial) oferă posibilitatea modificării variabilei de la final, ceea ce permite de noi elemente în aceeași structură. Iterarea peste elementele structurii incomplete diferă față de structurile complete prin condiția de terminare. Pentru structurile incomplete, în condiția de terminare, trebuie să se verifice dacă s-a ajuns la o variabilă neinițializată (cu predicatul *var*). Această clauză trebuie să fie prima din cadrul predicatului astfel încât să nu se facă o unificare nedorită.

Exemple:

```
?- L = [a, b, c|_].
?- L = [1, 2, 3|T], T = [4, 5|U].
?- T = t(7, t(5, t(3, _, _), _), t(11, _, _)).
?- T = t(7, t(5, t(3, A, B), C), t(11, D, E)), D = t(9, F, G).
```

2.2 Liste incomplete

2.2.1 Predicatul „member”

Înainte de a scrie noul predicat, urmăriți execuția la vechiul *member*:

```
?- L = [1, 2, 3|_], member1(3, L).
?- L = [1, 2, 3|_], member1(4, L).
?- L = [1, 2, 3|_], member1(X, L).
```

Noul predicat rezolvă problemele de la vechiul predicat.

% trebuie testat explicit faptul ca am ajuns la sfârșitul listei

% și nu am găsit elementul căutat

```
member_il(_, L):-var(L), !, fail.
```

% celelalte 2 clauze sunt la fel ca în trecut

```
member_il(X, [X|_]):-!.
```

```
member_il(X, [_|T]):-member_il(X, T).
```

Urmărește execuția la noul predicat pe aceleași întrebări.

2.2.2 Predicatul „insert”

După cum ați observat, adăugare unui element la sfârșitul unei liste se poate realiza pe lista de intrare și nu mai este nevoie de un parametru nou pentru lista rezultat.

```
% am ajuns la finalul listei atunci putem adăuga elementul
insert_il(X, L):-var(L), !, L=[X|_].
insert_il(X, [X|_]):-!. % elementul există deja
insert_il(X, [_|T]):- insert_il(X, T).
```

Funcționalitatea primei clauze poate fi realizată și de a doua clauză. Dacă elementul căutat nu a fost găsit înseamnă că am ajuns la finalul listei. Caz în care variabila din coada se va unifica cu `[X|_]`, ceea ce înseamnă că se inserează elementul căutat la sfârșitul listei.

Urmărește execuția la:

```
?- L = [1, 2, 3|_], insert_il(3, L).
?- L = [1, 2, 3|_], insert_il(4, L).
?- L = [1, 2, 3|_], insert_il(X, L).
```

2.2.3 Predicatul „delete”

În predicatul *delete* vom păstra aceeași variabilă neinițializată de la final și pentru lista de intrare și pentru lista rezultat. În rest predicatul este la fel cu varianta de la liste complete.

```
delete_il(_, L, L):-var(L), !. % am ajuns la finalul listei
delete_il(X, [X|T], T):-!. % ștergem prima apariție și ne oprim
delete_il(X, [H|T], [H|R]):-delete_il(X, T, R).
```

Urmărește execuția la:

```
?- L = [1, 2, 3|_], delete_il(2, L, R).
?- L = [1, 2, 3|_], delete_il(4, L, R).
?- L = [1, 2, 3|_], delete_il(X, L, R).
```

2.3 Arbori incompleți

2.3.1 Predicatul „search”

La fel ca și la predicatul *member* de la liste incomplete trebuie să verificăm explicit că am ajuns la o variabilă neinițializată.

```
search_it(_, T):-var(T),!,fail.
search_it(Key, t(Key, _, _)):-!.
search_it(Key, t(K, L, _)):-Key<K, !, search_it(Key, L).
search_it(Key, t(_, _, R)):-search_it(Key, R).
```

Urmărește execuția la:

```
?- T=t(7, t(5, t(3,_,_), t(6,_,_)), t(11,_,_)), search_it(6, T).  
?- T=t(7, t(5, t(3,_,_), _), t(11,_,_)), search_it(9, T).
```

2.3.2 Predicatul „insert”

Deoarece inserarea unui nod nou se realizează în frunze, ne putem folosi de același arbore de intrare și pentru rezultat. Ceea ce înseamnă că nu mai avem nevoie de încă un parametru pentru arborele rezultat.

% inserează sau verifică dacă elementul există deja în arbore

```
insert_it(Key, t(Key, _, _)):-!.  
insert_it(Key, t(K, L, R)):-Key<K, !, insert_it(Key, L).  
insert_it(Key, t(_, _, R)):-insert_it(Key, R).
```

Urmărește execuția la:

```
?- T=t(7, t(5, t(3,_,_), t(6,_,_)), t(11,_,_)), insert_it(6, T).  
?- T=t(7, t(5, t(3,_,_), _), t(11,_,_)), insert_it(9, T).
```

2.3.3 Predicatul „delete”

La fel ca în cazul predicatului *delete* de la liste incomplete vom păstra aceeași variabilă neinițializată din arborele de intrare și pe arborele rezultat.

```
delete_it(Key, T, T):-var(T),!.  
delete_it(Key, t(Key, L, R), L):-var(R),!.  
delete_it(Key, t(Key, L, R), R):-var(L),!.  
delete_it(Key, t(Key, L, R), t(Pred,NL,R)):-!,get_pred(L,Pred,NL).  
delete_it(Key, t(K,L,R), t(K,NL,R)):-Key<K,!,delete_it(Key,L,NL).  
delete_it(Key, t(K,L,R), t(K,L,NR)):-delete_it(Key,R,NR).
```

% caută nodul predecesor

```
get_pred(t(Pred, L, R), Pred, L):-var(R),!.  
get_pred(t(Key, L, R), Pred, t(Key, L, NR)):-get_pred(R, Pred, NR).
```

Urmărește execuția la:

```
?- T=t(7, t(5, t(3,_,_), t(6,_,_)), t(11,_,_)), delete_it(6, T, R).  
?- T=t(7, t(5, t(3,_,_), _), t(11,_,_)), delete_it(9, T, R).
```

3 Exerciții

Scrieți un predicat care:

1. Concatenează 2 liste incomplete (rezultatul este tot o listă incompletă).
2. Inversează o listă incompletă (rezultatul este tot o listă incompletă).
3. Convertește o listă incompletă într-o listă completă și viceversa.
4. Traversează un arbore incomplet în pre-ordine (cheile se adaugă într-o listă incompletă).
5. Calculează înălțimea unui arbore incomplet.
6. Convertește un arbore incomplet într-un arbore complet și viceversa.
7. Aplatizează o listă adâncă incompletă (rezultatul este o listă simplă incompletă).

```
?- flat_il([[1|_], 2, [3, [4, 5|_|_|_|_], R]).  
R = [1, 2, 3, 4, 5|_] ? ;  
false
```

8. Calculează diametrul unui arbore incomplet.

$$diam(T) = \max\{diam(T.left), diam(T.right), height(T.left) + height(T.right) + 1\}$$

9. (*) Determină dacă o listă incompletă este o sub-listă într-o altă listă incompletă.

```
?- subl_il([1, 1, 2|_], [1, 2, 3, 1, 1, 3, 1, 1, 1, 2|_]).  
true
```

```
?- subl_il([1, 1, 2|_], [1, 2, 3, 1, 1, 3, 1, 1, 1, 3, 2|_]).  
false
```

Liste diferență. Efecte laterale

1 Obiective

În lucrarea de față vom prezenta un nou tip de reprezentare pentru liste. Listele incomplete au fost un pas intermediar spre liste diferență. Dacă în listele incomplete, variabila din coadă este anonimă, în cazul listelor diferență variabila este dată într-un nou parametru. Dacă în listele complete/incomplete pentru a adăuga un element la final trebuie să parcurgem toată lista, în cazul listelor diferență se poate adăuga direct în variabila din coada listei.

2 Considerații teoretice

2.1 Reprezentare

Listele diferență se reprezintă prin două părți: începutul listei și sfârșitul listei. De exemplu, lista $L=[1,2,3]$ poate fi reprezentată de variabilele $S=[1,2,3|X]$ și $E=X$. Denumirea de „diferență” vine de la faptul că lista L poate fi calculată prin diferența dintre S și E . Lista vidă este reprezentată prin 2 variabile egale.

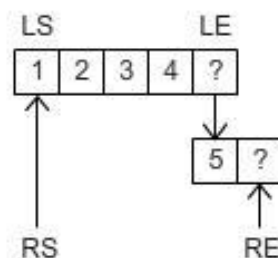
2.2 Predicatul „add”

În cazul listelor complete, adăugarea unui element la finalul listei se realizează în felul următor:

```
add1(X, [H|T], [H|R]):- add1(X, T, R).  
add1(X, [], [X]).
```

În cazul listelor diferență, predicatul se va scrie:

```
add2(X, LS, LE, RS, RE):- RS = LS, LE = [X|RE].  
% variabila de la finalul listei va conține pe prima poziție  
elementul de adăugat
```



Figură 1. Adăugarea unui element la sfârșitul unei liste diferență

Dacă îl testăm în interpretorul de Prolog ne va da următorul răspuns:

```
?- LS=[1,2,3,4|LE], add2(5,LS,LE,RS,RE).  
LE = [5|RE],  
LS = [1,2,3,4,5|RE],  
RS = [1,2,3,4,5|RE]
```

2.3 Predicatul „append”

În cazul listelor diferență, predicatul *append* se va scrie pe o singură linie:

```
append_dl(LS1,LE1, LS2,LE2, RS,RE):- RS=LS1, LE1=LS2, RE=LE2.
```

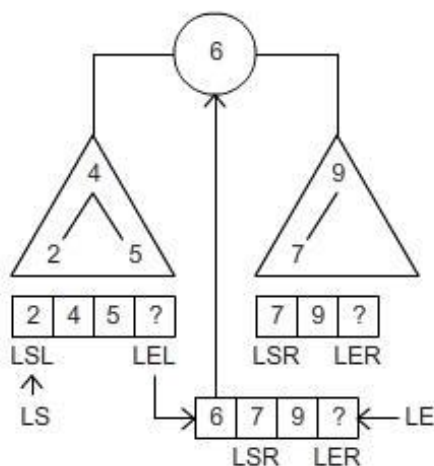
Exemple de utilizare:

2.3.1 Traversare în ordine

Putem îmbunătăți eficiența predicatului *inorder* prin înlocuirea vechiului *append* cu operații pe liste diferență.

```
inorder_dl(nil,L,L). % lista vida este reprezentată de 2 variabile egale
```

```
inorder_dl(t(K,L,R),LS,LE):-  
    inorder_dl(L,LSL,LEL), % apel pe subarbore stâng  
    inorder_dl(R,LSR,LER), % apel pe subarbore drept  
    LS=LSL,  
    LEL=[K|LSR], % K este adăugat în fața la LSR  
    LE=LER.
```



Figură 2. Traversarea în ordine

Urmărește execuția la:

```
?- tree1(T), inorder_dl(T,L,[]).
```

```
?- tree1(T), inorder_dl(T,L,_).
```

2.3.2 Sortare rapidă

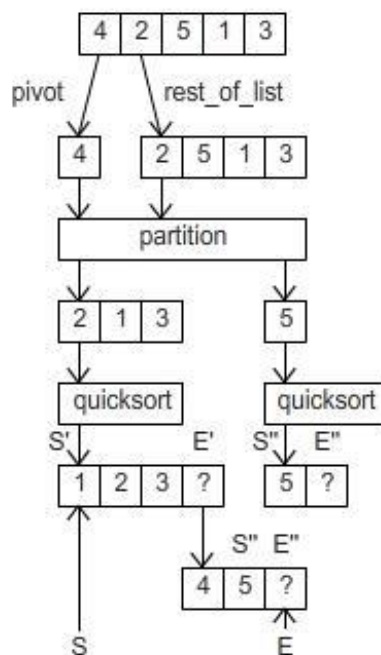
La fel ca mai sus, vom înlocui vechiul *append* cu operații pe liste diferență.

```
quicksort_dl([H|T], S, E):- % s-a adăugat un parametru nou  
    partition(H, T, Sm, Lg), % predicatul partition a rămas la fel  
    quicksort_dl(Sm, S, [H|L]),  
    quicksort_dl(Lg, L, E).  
quicksort_dl([], L, L). % condiția de terminare s-a modificat
```

Urmărește execuția la:

?- quicksort_d1([4,2,5,1,3], L, []).

?- quicksort_d1([4,2,5,1,3], L, _).



Figură 3. Sortare rapidă folosind liste diferență

2.4 Efecte laterale

Efectele laterale se referă la manipularea dinamică a definițiilor de predicate și acest lucru se poate realiza cu următoarele predicate predefinite:

- *assert/1* = *assertz/1* = adaugă la sfârșitul bazei de cunoștințe clauza dată în argument;
- *asserta/1* = adaugă la începutul bazei de cunoștințe clauza dată în argument;
- *retract/1* = șterge prima clauză care se unifică cu argumentul;
- *retractall/1* = șterge toate clauzele care se unifică cu argumentul (în SWI-Prolog va merge la succes chiar dacă nu șterge nimic).

Predicatele care sunt definite și/sau apelate în fișierul „.pl” se numesc predicate statice. Predicatele care sunt manipulate cu *assert/retract* se numesc predicate dinamice. Dacă un predicat este adăugat pentru prima dată cu *assert* atunci este implicit un predicat dinamic. Dacă vrem să manipulăm un predicat care apare în fișierul „.pl” atunci trebuie să-l setăm ca și predicat dinamic adăugând următoare linie la începutul fișierului:

```
:-dynamic nume_predicat/aritate.
```

Când folosim aceste predicate trebuie să ținem cont de următoarele aspecte:

- Backtracking-ul nu invalidează efectul la *assert* (ex: dacă un predicat a fost adăugat cu *assert*, el poate fi șters numai cu *retract*);
- În cazul predicatului *retract*, backtracking-ul invalidează temporar ștergerea pentru predicatele din același corp al clauzei cu apelul predicatului *retract* și care au fost apelate înaintea predicatului *retract*; astfel se păstrează „logical update view”;
- Predicatul *assert* merge tot timpul la succes;
- Predicatul *retract* poate să eșueze, caz în care va porni backtracking-ul.

Pentru a înțelege mai bine cum funcționează *retract* urmăriți execuția la:

```
?- assert(insect(ant)),
    assert(insect(bee)),
    retract(insect(A)),
    writeln(A),
    retract(insect(B)),
    fail.
```

Deși al doilea *retract* șterge faptul *insect(bee)*, când se face backtracking, primul *retract*, mai exact *insect(A)* nu vede ștergerea lui „bee” și va face unificarea cu *insect(bee)*.

Manipularea dinamică a bazei de cunoștințe este folosită în special pentru salvarea rezultatelor de la calcule (memoisation/caching). De asemenea aceste operații pot fi folosite pentru a schimba dinamic comportamentul unor predicate la rulare (meta-programming).

Exemplu de predicat care memorizează rezultatele parțiale:

```
:-dynamic memo_fib/2.
```

```
fib(N,F):- memo_fib(N,F), !.
fib(N,F):- N>1,
           N1 is N-1,
           N2 is N-2,
           fib(N1,F1),
           fib(N2,F2),
           F is F1+F2,
           assertz(memo_fib(N,F)).

fib(0,1).
fib(1,1).
```


Urmărește execuția la:

```
?- listing(memo_fib/2). % afișează toate definițiile predicatului
memo_fib cu 2 parametrii
?- fib(4,F).
?- listing(memo_fib/2).
?- fib(10,F).
?- listing(memo_fib/2).
?- fib(10,F).
```

2.4.1 Afișarea rezultatelor memorizate

Ne vom folosi de tehnica backtracking pentru a parcurge toate predicatele adăugate în baza de cunoștințe cu *assert*.

```
print_all:-memo_fib(N,F),
            write(N),
            write(' - '),
            write(F),
            nl,
            fail.
print_all.
```

Urmărește execuția la:

```
?-print_all.
?-retractall(memo_fib(_,_)).
?-print_all.
```

2.4.2 Colectarea rezultatelor memorizate

Pentru colectarea rezultatelor într-o listă ne putem folosi de un predicat predefinit: *findall*.

Urmărește execuția la:

```
?- findall(X, append(X,_,[1,2,3,4]), List).
?- findall(lists(X,Y), append(X,Y,[1,2,3,4]), List).
?- findall(X, member(X,[1,2,3]), List).
```

Ne vom folosi de efecte laterale pentru a colecta toate permutările unei liste de elemente.

```
all_perm(L,_):- perm(L,L1), % predicatul de generare a unei
permutări (vezi lucrare de laborator cu sortări)
                assertz(p(L1)),
                fail.
all_perm(_,R):- collect_perms(R).
```

```
collect_perms([L1|R]):- retract(p(L1)), !, collect_perms(R).
collect_perms([]).
```

Urmărește execuția la:

```
?- retractall(p(_)),all_perm([1,2],R).
?- listing(p/1).
?- retractall(p(_)),all_perm([1,2,3],R).
```

Întrebări:

1. De ce am nevoie de *retractall* înainte de apelul la *all_perm*?
2. De ce este nevoie de ! după *retract* în predicatul *collect_perms*?
3. Ce tip de recursivitate este folosit în predicatul *collect_perms*?
4. Predicatul *collect_perms* distruge rezultatele salvate?

3 Exerciții

Scrieți un predicat care:

1. Convertește o listă incompletă într-o listă diferență și viceversa.
2. Convertește o listă completă într-o listă diferență și viceversa.
3. Generează toate descompunerile posibile a unei liste în doua sub-liste fără a folosi predicatul predefinit *findall*.

```
?- all_decompositions([1,2,3], List).
List= [ [], [1,2,3]], [[1], [2,3]], [[1,2], [3]], [[1,2,3], []] ;
false
```

4. Aplatizează o listă adâncă folosind liste diferență în loc de *append*.

```
?- flat_d1([[1], 2, [3, [4, 5]]], RS, RE).
RS = [1, 2, 3, 4, 5|RE] ;
false
```

5. Colectează toate nodurile care au chei pare, dintr-un arbore binar folosind liste diferență.
6. Colectează toate nodurile care au chei între K1 și K2, dintr-un arbore binar folosind liste diferență.

Grafuri

1 Obiective

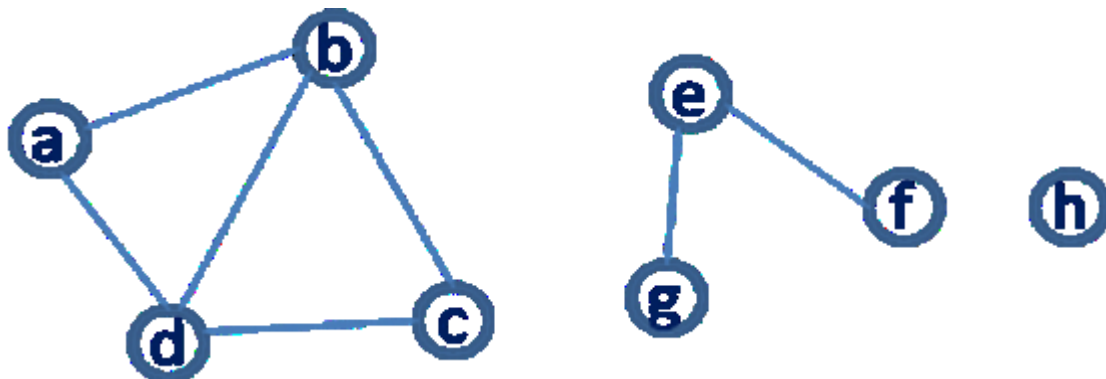
În lucrarea de față vom prezenta cum se poate reprezenta un graf în Prolog și cum putem căuta drumul între două noduri.

2 Considerații teoretice

2.1 Reprezentare

Un graf este reprezentat de două mulțimi: noduri și muchii. Dacă graful este orientat atunci vorbim despre: vârfuri și arce.

Exemplu de graf neorientat:



Există mai multe alternative de a reprezenta un graf în Prolog și le putem clasifica după următoarele criterii:

A. Componentele reprezentate

1. Noduri și muchii;
2. Noduri și lista asociată de vecini;

B. Locul unde sunt salvate

1. Într-o structură;
2. Într-o colecție de predicate de tip axiomă.

A1B2: O colecție de predicate *node* și *edge* (edge-clause)

```
node(a). node(b). %etc  
edge(a,b). edge(b,a).  
edge(b,c). edge(c,b).  
%etc
```

Avem nevoie de predicatele de tip *node* pentru cazurile în care avem noduri izolate.

A2B2: O colecție de predicate *neighbor* (neighbor list-clause)

```
neighbor(a, [b, d]).
neighbor(b, [a, c, d]).
neighbor(c, [b, d]).
neighbor(h, []).
%etc
```

A1B1: O pereche formată dintr-o listă de noduri și o listă de muchii (graph-term)

```
?- Graph = graph([a,b,c,d,e,f,g,h], [e(a,b), e(b,a), ... ]).
```

A2B1: O listă de perechi: nod, listă asociată de vecini (neighbor list-list)

```
?- Graph = [n(a, [b,d]), n(b, [a,c,d]), n(c, [b,d]), n(d, [a,b,c]), n(e,
[f,g]), n(f, [e]), n(g, [e]), n(h, [])].
```

Reprezentarea care va fi folosită depinde de cerințele problemei. Mai jos este dat un exemplu de conversie între A2B2 în A1B2.

```
:-dynamic neighbor/2. % declarăm predicatul dinamic pentru a putea
folosi retract
```

```
neighbor(a, [b, d]).
neighbor(b, [a, c, d]).
neighbor(c, [b, d]).
%etc.
```

```
neighb_to_edge:-retract(neighbor(Node,List)),!, %extrage un predicat
neighbor și apoi îl procesează
                process(Node,List),
                neighb_to_edge.
neighb_to_edge. % daca nu mai sunt predicate neighbor înseamnă că am
terminat
```

```
% procesarea presupune adăugare de predicate edge și node pentru un
predicat neighbor
```

```
process(Node, [H|T]):- assertz(edge(Node, H)), process(Node, T).
process(Node, []):- assertz(node(Node)).
```

2.2 Drumuri în graf

2.2.1 Simple

Presupunem că graful este reprezentat prin predicate *node* și *edge*. Următorul predicat caută un drum între două noduri și returnează lista de noduri parcurse.

```

% path(Source, Target, Path)
path(X,Y,Path):-path(X,Y,[X],Path). % drumul parțial începe cu
punctul de pornire
path(Y,Y,PPath, PPath). % când ținta este egală cu
sursa am terminat
path(X,Y,PPath, FPath):-
    edge(X,Z), % căutăm o muchie
    not(member(Z, PPath)), % care nu a mai fost parcursă
    path(Z, Y, [Z|PPath], FPath). % și o adăugăm în rezultatul
parțial

```

Urmărește execuția la:

```
?- path(a,c,R).
```

2.2.2 Restricționate

Drumul restricționat presupune trecerea printr-un anumit set de noduri în ordinea dată. Deoarece *path* returnează drumul în ordine inversă atunci vom aplica reverse.

```

% restricted_path(Source, Target, RestrictionsList, Path)
restricted_path(X,Y,LR,P):- path(X,Y,P), reverse(P,PR),
    check_restrictions(LR, PR).

% verificăm dacă se respectă restricția
check_restrictions([],_):- !.
check_restrictions([H|T], [H|R]):- !, check_restrictions(T,R).
check_restrictions(T, [H|L]):-check_restrictions(T,L).

```

Urmărește execuția la:

```
?- restricted_path(a, c, [a,c,d], R).
```

2.2.3 Optimale

Considerăm un drum optim între două noduri, acel drum care are lungimea minimă.

```

% optimal_path(Source, Target, Path)
optimal_path(X,Y,Path):-
    asserta(sol_part([],100)), % distanța maximă inițială
    path(X,Y,[X],Path,1).
optimal_path(_,_,Path):-
    retract(sol_part(Path,_)).

path(Y,Y,Path,Path,LPath):- % când ținta este egală cu
sursa, salvăm soluția curentă
    retract(sol_part(_,_)),!,
    asserta(sol_part(Path,LPath)),
fail. % și căutăm o altă soluție

```

```

path(X,Y,PPath,FPath,LPath):-
    edge(X,Z),
    not(member(Z,PPath)),
    LPath1 is LPath+1,           % calculăm distanța parțială
    sol_part(_,Lopt),           % extragem distanța de la
    soluția precedentă
    LPath1<Lopt,                % dacă distanța curentă nu
    depășește distanța precedentă mergem mai departe
    path(Z,Y,[Z|PPath],FPath,LPath1).

```

Urmărește execuția la:

```
?- optimal_path(a,c,R).
```

2.2.4 Ciclu Hamiltonian

Un ciclu Hamiltonian este un ciclu care trece prin toate nodurile o singură dată (cu excepția nodului de start care este începutul și sfârșitul acestui ciclu).

```
% hamilton(NumOfNodes, Source, Path)
```

```
hamilton(NN, X, Path):- NN1 is NN-1, hamilton_path(NN1,X, X,
[X],Path).
```

3 Exerciții

1. Scrieți un predicat care convertește din A1B2 (edge-clause) în A2B2 (neighbor list-clause).
2. Continuă implementarea la ciclul Hamiltonian.
3. Rescrie *optimal_path* folosind ponderea de pe muchie (predicatul *edge* va avea 3 parametrii).
4. Scrie un predicat care găsește un ciclu ce pornește din nodul X și pune rezultatul în R.

```
?- cycle(a, R).
```

```
R = [a,b,c,d,a] ;
```

```
R = [a,b,d,a] ;
```

```
false
```

5. (*) Scrie o serie de predicate care să rezolve problema „Lupul-Capra-Varza”: un fermier trebuie să mute în siguranță, de pe malul nordic pe malul sudic, un lup, o capră și o varză. În barcă încap maxim doi și unul dintre ei trebuie să fie fermierul. Dacă fermierul nu este pe același mal cu lupul și capra, lupul va mânca capra. Același lucru se întâmplă și cu varza și capra, capra va mânca varza.

Sugestii:

- a. reprezentați lumea printr-o listă cu poziția celor 4 obiecte [Fermier, Lup, Capra, Varza];
- b. starea inițială este [n,n,n,n] (toți sunt în nord) și starea finală este [s,s,s,s] (toți au ajuns în sud);
- c. la fiecare trecere fermierul își schimbă poziția (din „n” în „s” sau invers) și **cel mult** încă un participant;
- d. problema poate fi văzută ca o problema de căutare a drumului într-un graf.

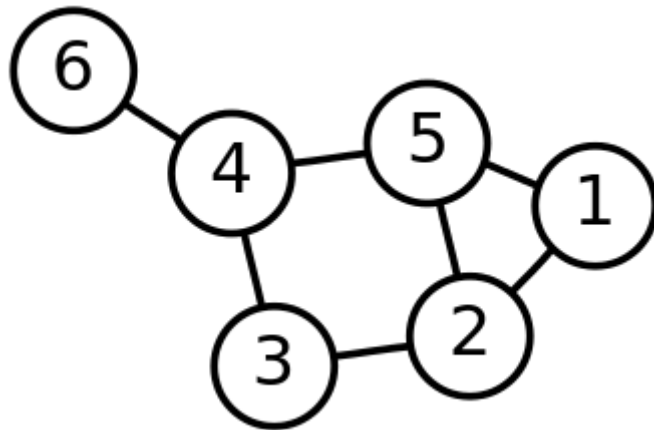
Parcurgerea în adâncime și în lățime

1 Obiective

În lucrarea de față vom prezenta cum se pot parcurge grafurile în Prolog. Grafurile le vom reprezenta prin predicate edge.

2 Considerații teoretice

Exemplu de graf neorientat:



În Prolog:

```
edge(1,5).  
edge(1,2). %etc
```

2.1 Parcurgere în adâncime (DFS)

Ne vom folosi de un predicat auxiliar pentru a stoca nodurile deja vizitate.

```
:- dynamic nod_vizitat/1.
```

% d_search(Source, Path)

```
d_search(X,_):-df_search(X,_). % parcurgerea nodurilor
```

```
d_search(_,L):-!, collect_reverse([],L). % colectarea rezultatelor
```

```
df_search(X,L):-  
    asserta(nod_vizitat(X)),  
    edge(X,Y),  
    not(nod_vizitat(Y)),  
    df_search(Y,L).
```

% colectarea se face în ordine inversa

```
collect_reverse(L,P):-  
    retract(nod_vizitat(X)), !,  
    collect_reverse([X|L],P).  
collect_reverse(L,L).
```


Urmărește execuția la:

?- d_search(1,R).

2.2 Parcurgerea în lățime (BFS)

În cazul parcurgerii în lățime avem nevoie de un predicat care să ne spună care este următorul nod de expandat. În predicatul „nod_vizitat” vom stoca elementele în ordine parcurgerii lor.

% b_search(Source, Path)

b_search(X,_):- % parcurgerea nodurilor

 assertz(nod_vizitat(X)),

 assertz(nod_de_expandat(X)),

 bf_search.

b_search(_,R):-!, collect_reverse([],R). % colectarea rezultatelor

bf_search:-

 retract(nod_de_expandat(X)),

 expand(X),!,

 bf_search.

expand(X):-

 edge(X,Y),

 not(nod_vizitat(Y)),

 asserta(nod_vizitat(Y)),

 assertz(nod_de_expandat(Y)),

 fail.

expand(_).

Urmărește execuția la:

?- b_search(1,R).

3 Exerciții

1. Modificați predicatul DFS astfel încât să caute noduri numai până la o anumită adâncime (DLS – Depth-Limited Search).